



UNIVERSITÀ DEGLI STUDI DI SALERNO

Dipartimento di Ingegneria dell'Informazione ed Elettrica e Matematica Applicata

Corso di Laurea Magistrale in Ingegneria Informatica

ELABORATO PER L'INSEGNAMENTO DI

Algoritmi e Protocolli per la Sicurezza

Progettazione di un Sistema di Voto Elettronico Sicuro

Premio "Tesi dell'Anno" dell'Ateneo

DOCENTE

Prof. Carlo Mazzocca

CANDIDATI

Errico Gioia
Raffaele Cutolo

Anno Accademico 2025/2026

WP 1 — Modello

Sistema di voto elettronico sicuro

Premio “Tesi dell’Anno” dell’Ateneo

1.1 Scenario e Funzionalità del Sistema

La transizione verso infrastrutture di voto elettronico (*e-voting*) per consultazioni ad alto valore istituzionale rappresenta una sfida ingegneristica complessa, caratterizzata da un intrinseco paradosso crittografico: la necessità di bilanciare la trasparenza matematica del conteggio con l’assoluta segretezza della singola preferenza. Un sistema concepito come *black-box* può offrire una privacy teorica perfetta, ma preclude la verificabilità pubblica, aprendo la strada a dubbi sulla legittimità dell’esito; viceversa, un sistema puramente trasparente distrugge l’anonimato, abilitando ritorsioni e compravendita dei voti.

Il presente progetto ha come obiettivo la definizione del modello per la conduzione di un’elezione elettronica sicura, specificatamente progettata per l’assegnazione del Premio “Tesi dell’Anno” dell’Ateneo. Il premio viene assegnato annualmente dalla comunità studentesca, che elegge — tramite votazione — la miglior tesi di laurea tra un insieme di candidature preventivamente selezionate e approvate da una commissione accademica.

La tipologia di elezione scelta è quella con **lista chiusa**: gli aventi diritto al voto devono esprimere la propria preferenza per una singola tesi candidata, scelta all’interno di un insieme predefinito e finito di opzioni. Oltre alla preferenza per le singole tesi, il sistema ammette una **scheda bianca** deliberata come opzione legittima. Questa scelta è motivata sia da ragioni funzionali — rispecchia fedelmente il formato della votazione — sia da ragioni crittografiche: uno spazio di voti piccolo e finito consente di adottare tecniche più efficienti rispetto a una preferenza nominale libera, riducendo il costo computazionale sia lato client che lato server. La natura accademica e socialmente densa del contesto rende inoltre particolarmente critici i requisiti di segretezza e resistenza alla coercizione: trattandosi di votare tesi di colleghi e conoscenti, la possibilità di pressioni informali o di acquisto del voto è concreta e non trascurabile. L’efficienza è considerata un vincolo progettuale fin da questa fase: il sistema deve risultare utilizzabile da un comune dispositivo consumer (smartphone o PC), senza latenze eccessive nelle operazioni di voto e verifica.

L’obiettivo primario è la conduzione di questa elezione in modo da garantire **autenticità, segretezza, integrità e verificabilità** (individuale e universale). Il sistema deve raccogliere i voti in modo che l’identità dell’elettore rimanga separata dalla sua preferenza (garantendo lo *pseudoanonimato*), permettendo al contempo alla commissione organizzatrice di pubblicare il risultato finale dimostrandone la correttezza.

Sebbene la letteratura offra sofisticati modelli di *weighted voting* (voto pesato) in cui il voto di un professore ordinario potrebbe teoricamente valere più di quello di un ricercatore o di uno studente, tali schemi richiedono protocolli crittografici complessi, che appesantiscono drasticamente la complessità computazionale. Per rispettare i rigorosi vincoli di efficienza computazionale richiesti dal sistema, il sistema adotterà un approccio egualitario (1 elettore = 1 voto).

1.2 Attori Onesti del Sistema e Obiettivi

Per mitigare i rischi legati alla concentrazione del potere, il sistema prevede la separazione dei ruoli tra le seguenti entità, ciascuna delle quali è assunta come *onesta* nell’esercizio delle proprie funzioni:

- **Elettori (Membri della Comunità Accademica):** gli utenti legittimi aventi diritto di voto, ovvero gli studenti regolarmente iscritti e i docenti afferenti all'Ateneo. Entrambe le categorie esprimono un voto con peso identico: non è prevista alcuna distinzione nel conteggio finale tra il voto di uno studente e quello di un docente. Il loro obiettivo è esprimere la propria preferenza in modo anonimo, con la garanzia che il proprio voto venga correttamente registrato e conteggiato. La natura socialmente densa del contesto — in cui i candidati sono colleghi e conoscenti di entrambe le categorie — rende concreta la possibilità di pressioni informali tra pari o tra docenti e studenti, rafforzando ulteriormente la necessità di segretezza e resistenza alla coercizione.
- **Commissione Accademica:** il soggetto responsabile della selezione preliminare delle tesi proposte per il premio. Il suo obiettivo è definire l'insieme finito e ufficiale delle opzioni di voto, garantendo che solo le candidature legittime e approvate siano incluse nella scheda elettorale. La Commissione Accademica opera esclusivamente nella fase pre-elettorale e non partecipa alla raccolta né allo scrutinio dei voti, riducendo così il rischio di influenza sul risultato finale.
- **Sistema di Autenticazione (Identity Provider — IdP):** l'ente incaricato di accertare l'identità degli elettori attraverso le proprie credenziali. Il suo obiettivo è rilasciare un'autorizzazione al voto valida, assicurandosi che solo gli aventi diritto — studenti iscritti o docenti afferenti all'Ateneo — partecipino e che nessuno possa votare più di una volta. *L'IdP non deve essere in grado di conoscere il contenuto del voto dell'elettore che sta autorizzando.*
- **Autorità Elettorale:** l'entità responsabile della raccolta dei voti e dello scrutinio finale. Il suo obiettivo è registrare i voti in modo integro, eseguire il conteggio in modo trasparente e pubblicare il risultato finale senza mai venire a conoscenza dell'identità reale del singolo votante associata a una specifica scheda. *L'Autorità Elettorale non deve essere in grado di autenticare autonomamente gli elettori*, essendo questa funzione demandata esclusivamente all'IdP.

Assunzione di fiducia fondamentale. Il corretto funzionamento del sistema in termini di pseudoanonimato dipende dall'assunzione che *IdP e Autorità Elettorale non si mettano d'accordo tra loro*. L'IdP conosce l'identità dell'elettore ma non il contenuto del voto; l'Autorità Elettorale conosce il voto ma non l'identità di chi lo ha espresso. Nessuna delle due entità, singolarmente, è in grado di violare la segretezza. Questa separazione è una scelta progettuale deliberata: introduce complessità architetturale ma è il principale meccanismo di tutela dello pseudoanonimato. Il rischio residuo di **collusione** (accordo) tra le due entità è riconosciuto come limitazione del modello e verrà analizzato nei *work package* successivi.

La Figura 1 mostra il diagramma dell'architettura degli attori, le loro relazioni, i flussi principali (autenticazione, token, voto cifrato, ricevuta) e la barriera visiva di non collusione tra IdP e Autorità Elettorale, con la nota che ciascuno conosce solo metà dell'informazione.

1.3 Modello di Minaccia (Threat Model)

Il sistema deve essere progettato per resistere ad avversari con capacità e risorse differenti, interessati a compromettere l'elezione. Si identificano i seguenti profili di minaccia:

- **Identity Provider (IdP) disonesto:** un avversario interno con elevate risorse computazionali e pieno accesso al sistema di autenticazione. I suoi obiettivi includono l'ammissione fraudolenta di elettori non aventi diritto, la soppressione selettiva di autorizzazioni al voto per penalizzare determinati candidati, o il tentativo di tracciare gli elettori per correlare le loro identità ai voti espressi.

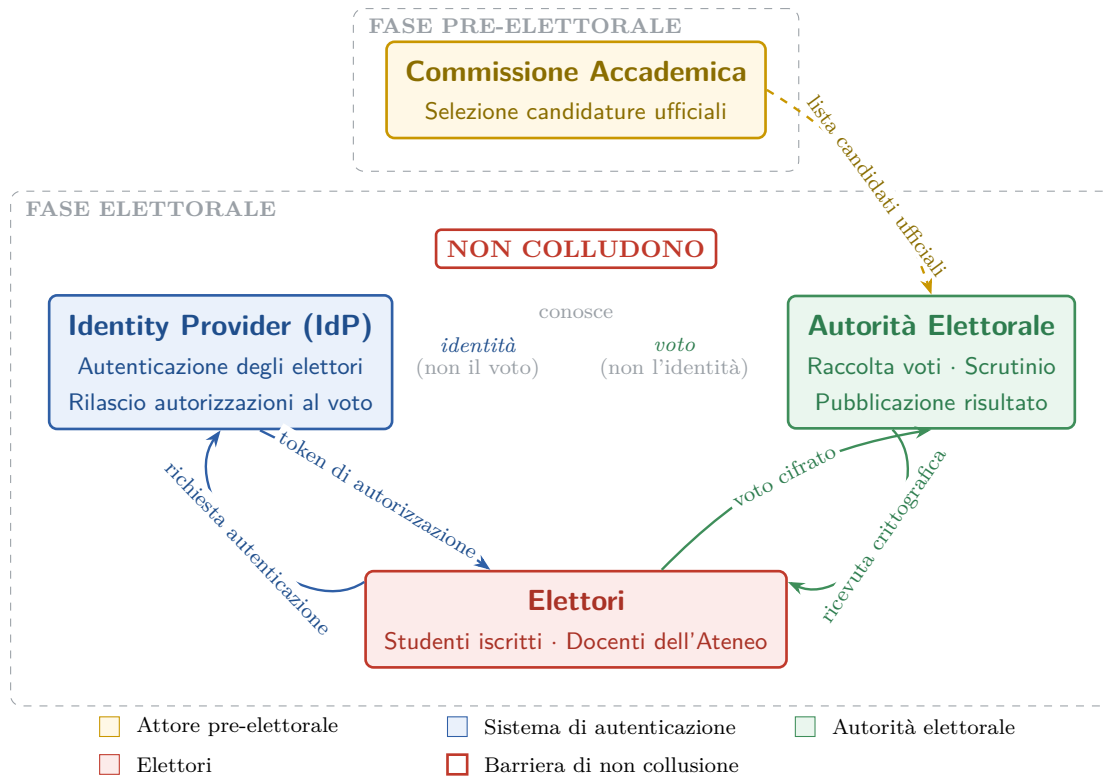


Figura 1: Architettura degli attori del sistema, flussi principali e barriera di non collusione tra IdP e Autorità Elettorale.

- **Autorità Elettorale disonesta:** un avversario interno con elevate risorse computazionali e pieno accesso all'urna elettorale. I suoi obiettivi includono l'alterazione del conteggio finale a favore di una tesi candidata, la soppressione di voti legittimi, l'introduzione fraudolenta di voti, o il tentativo di violare la segretezza delle schede per identificare i votanti.
- **Collusione tra IdP e Autorità Elettorale:** lo scenario più critico per la segretezza del sistema. Se IdP e Autorità Elettorale cooperano in modo malevolo, combinando le informazioni in loro possesso — rispettivamente, l'identità dell'elettore e il contenuto del voto — possono ricostruire la corrispondenza completa tra votante e preferenza, violando lo pseudoanonimato. Questo scenario è particolarmente insidioso perché nessuna delle due entità, presa singolarmente, dispone di informazioni sufficienti a violare la segretezza: la minaccia emerge esclusivamente dalla loro cooperazione malevola. Il rischio residuo associato a questo scenario è riconosciuto come limitazione strutturale del modello adottato.
- **Elettore malintenzionato:** un avversario con risorse computazionali standard (es. un personal computer o uno smartphone). Tenta di aggirare le regole dell'elezione cercando di votare più di una volta, di ripudiare il proprio voto a posteriori, o di dimostrare a terzi in modo inequivocabile come ha votato per facilitare scenari di coercizione o compravendita del voto. Si precisa che la capacità dell'elettore di provare volontariamente la propria preferenza — e la conseguente vulnerabilità alla coercizione — non rientra tra le minacce contrastate dal sistema: è un limite accettato consapevolmente, conseguenza della verificabilità individuale, discusso in 1.5.
- **Avversario di rete (Eavesdropper / Man-in-the-Middle):** un attaccante esterno che intercetta i canali di comunicazione tra gli elettori e i server. Tenta di correlare i voti alle identità reali analizzando il traffico di rete, o di alterare i messaggi in transito per modificare voti o invalidare autorizzazioni. Dispone di risorse computazionali potenzialmente elevate e

di accesso all'infrastruttura di rete.

- **Avversario DoS (Denial of Service):** un attaccante — interno o esterno — che mira a compromettere la disponibilità del sistema durante la finestra temporale di votazione. I suoi obiettivi includono il rendere irraggiungibile l'urna elettorale o il sistema di autenticazione, impedendo agli aventi diritto di esprimere il proprio voto entro i termini stabiliti. Sebbene questo tipo di attacco non violi direttamente la segretezza o l'integrità dei voti, può compromettere la regolarità e la completezza dell'elezione. La gestione di questo rischio non rientra nell'ambito degli strumenti crittografici discussi nel corso e, pertanto, non sarà oggetto di trattazione in questo progetto. Sebbene non affrontata, la sua menzione è necessaria ai fini della completezza del modello di minaccia.

Nota. Per “elevate risorse computazionali” si intende, chiaramente, un avversario probabilistico in tempo polinomiale (PPT).

La Tabella 1 contiene la matrice delle minacce, che mostra in modo immediato quale avversario attacca quale proprietà. Le celle rosse indicano una compromissione diretta, quelle arancioni un rischio parziale.

Avversario / Minaccia	Autenticità	Unicità	Segretezza	Integrità	Verif. Individuale	Verif. Universale	Disponibilità	Irrevocabilità
IdP disonesto	×	×	~	—	—	—	—	—
Autorità Elettorale disonesta	—	—	~	×	×	×	—	—
Collusione IdP + Autorità Elettorale	—	—	×	—	—	—	—	—
Elettore malintenzionato	—	×	~	—	—	—	—	×
Eavesdropper / Man-in-the-Middle	~	—	×	×	—	—	—	—
Avversario DoS	—	—	—	—	—	—	×	—

× Proprietà direttamente compromessa ~ Proprietà parzialmente a rischio — Proprietà non interessata

Tabella 1: Matrice delle minacce: relazione tra profili di avversario e proprietà di sicurezza.

Si vede chiaramente, ad esempio, che la collusione è l'unica minaccia che colpisce la segretezza in modo diretto e irrimediabile. Allo stesso modo, un elettore malintenzionato potrebbe colpire la proprietà di irrevocabilità, in quanto potrebbe voler votare più di una volta, provando a revocare il vecchio voto e, di conseguenza, violando la proprietà di irrevocabilità.

1.4 Proprietà di Sicurezza da Preservare

In presenza degli attacchi descritti nel *threat model*, il sistema progettato dovrà essere in grado di garantire le seguenti proprietà. Si riconosce esplicitamente che alcune di esse sono in tensione strutturale tra loro: il raggiungimento simultaneo e incondizionato di tutte le proprietà non è possibile, e i compromessi adottati vengono dichiarati e motivati ove rilevante.

- **Autenticità:** solo gli elettori legittimi — studenti regolarmente iscritti e docenti afferenti all'Ateneo — preventivamente autenticati tramite le proprie credenziali, possono partecipare alla procedura di voto. Qualsiasi tentativo di partecipazione da parte di soggetti non aventi diritto deve essere rilevato e respinto.

- **Unicità:** ciascun elettore autenticato ha il diritto di esprimere al massimo un voto valido. Il sistema deve impedire che uno stesso soggetto possa votare più di una volta, indipendentemente dal dispositivo o dal canale utilizzato. Si noti che questa proprietà è in tensione con lo pseudoanonimato: per verificare che un elettore non abbia già votato, il sistema necessita di tracciare in qualche forma l'avvenuta partecipazione, senza però rivelare il contenuto del voto. La gestione di questo compromesso è demandata alla separazione dei ruoli tra IdP e Autorità Elettorale descritta nella Sezione 1.2.
- **Segretezza (Pseudoanonimato del voto):** il voto del singolo elettore non deve essere rivelabile in alcun modo. Il sistema deve garantire che nessuna singola entità — né un attaccante esterno, né i gestori del sistema — sia in grado di associare direttamente l'identità reale dell'elettore alla sua preferenza, nemmeno combinando in modo malevolo le informazioni in possesso del Sistema di Autenticazione e dell'Autorità Elettorale. Si riconosce che questa proprietà è in tensione con la verificabilità individuale: consentire a un elettore di verificare il proprio voto richiede che egli disponga di informazioni sufficienti a identificare la propria scheda nell'urna, senza però che tali informazioni siano utilizzabili da terzi per violare la segretezza. Questo compromesso verrà affrontato nella progettazione del meccanismo di verifica individuale.
- **Integrità:** le preferenze espresse e registrate nell'urna non devono poter essere alterate, modificate, rimosse o introdotte fraudolentemente in momenti successivi alla loro espressione. Questa proprietà deve valere anche nei confronti dell'Autorità Elettorale stessa, che non deve poter manipolare i voti registrati senza che tale manipolazione sia rilevabile.
- **Verificabilità individuale:** ogni singolo elettore deve essere dotato degli strumenti necessari per controllare in modo del tutto autonomo che il proprio specifico voto sia stato correttamente ricevuto e incluso nel conteggio finale dell'Autorità Elettorale. Come discusso nella proprietà di segretezza, il meccanismo di verifica individuale deve essere progettato in modo da non compromettere lo pseudoanonimato dell'elettore.
- **Verificabilità universale:** il risultato finale dell'elezione deve essere pubblicamente verificabile da chiunque — inclusi osservatori esterni, candidati e commissione accademica — dimostrando la correttezza matematica dello scrutinio senza rivelare l'identità del votante.
- **Disponibilità:** il sistema deve essere raggiungibile e funzionante per tutta la durata della finestra temporale di votazione. Come dichiarato nella Sezione 1.3, la mitigazione degli attacchi DoS esula dagli strumenti crittografici trattati nel corso e non verrà affrontata nel presente progetto. La disponibilità è pertanto riconosciuta come proprietà desiderabile ma non garantita dalla soluzione proposta.
- **Irrevocabilità del voto:** si sceglie consapevolmente di non supportare la revoca o la sostituzione del voto una volta espresso. Questa scelta è motivata da tre considerazioni: in primo luogo, un voto espresso è un atto deliberato e definitivo, coerente con la prassi delle votazioni accademiche formali; in secondo luogo, l'irrevocabilità rafforza la resistenza alla coercizione, poiché nessun avversario può forzare un elettore a modificare un voto già registrato; in terzo luogo, la gestione crittograficamente sicura della revoca richiederebbe strumenti avanzati non trattati nel corso, introducendo complessità sproporzionata rispetto ai benefici nel contesto specifico. Si riconosce tuttavia che in contesti elettorali diversi, dove la possibilità di ripensamento è un requisito funzionale, questa scelta costituirebbe un limite del modello.

1.5 Riepilogo dei Compromessi Adottati

La progettazione di un sistema di voto elettronico sicuro impone di navigare tra proprietà che sono in parte strutturalmente conflittuali. Il presente modello adotta i seguenti compromessi, ciascuno dei quali è frutto di una scelta consapevole e motivata:

- **Segretezza vs. Unicità:** per garantire che nessun elettore voti più di una volta, il sistema deve tracciare in qualche forma l'avvenuta partecipazione. Questo è ottenuto affidando all'IdP la gestione delle autorizzazioni al voto, separandola completamente dalla gestione dei contenuti affidata all'Autorità Elettorale. Il compromesso è accettabile a patto che le due entità non colludano — assunzione dichiarata esplicitamente come rischio residuo del modello.
- **Segretezza vs. Verificabilità individuale:** consentire a un elettore di verificare il proprio voto richiede che egli disponga di un riferimento alla propria scheda nell'urna. Questo meccanismo deve essere progettato con cura per evitare che tale riferimento possa essere sfruttato da terzi per violare lo pseudoanonimato. La soluzione a questo compromesso è demandata alla fase di progettazione in WP2.
- **Completezza vs. Semplicità:** alcune proprietà desiderabili — la disponibilità del sistema e la revocabilità del voto — non vengono garantite dalla soluzione proposta. Nel caso della disponibilità, la mitigazione degli attacchi DoS esula dagli strumenti crittografici trattati nel corso. Nel caso della revocabilità, la scelta di non supportarla è motivata da ragioni di semplicità progettuale e di rafforzamento della resistenza alla coercizione. Entrambe le esclusioni sono riconosciute come limitazioni del modello e dichiarate esplicitamente come tali.
- **Trasparenza vs. Fiducia residua:** il sistema non è completamente *trustless* — si affida alla correttezza individuale di IdP e Autorità Elettorale, e al fatto che queste non collaborino. Questa assunzione è il principale punto di fragilità del modello e rappresenta la limitazione strutturale più rilevante.
- **Verificabilità individuale vs. Resistenza alla coercizione (receipt-freeness):** esiste una tensione strutturale tra due obiettivi. Da un lato consentire a ciascun elettore di verificare autonomamente che il proprio voto sia stato incluso nel conteggio (verificabilità individuale); dall'altro impedire che lo stesso elettore possa dimostrare a terzi come ha votato (*receipt-freeness*, alla base della resistenza alla coercizione). Qualunque capacità data all'elettore di riconoscere il proprio voto all'interno dell'urna può, in linea di principio, essere sfruttata dall'elettore stesso — volontariamente o sotto costrizione — per provare la propria scelta. Il presente modello dà priorità alla verificabilità individuale e riconosce esplicitamente che la resistenza alla coercizione non è garantita. Si sottolinea che ciò non intacca la segretezza del voto: sotto l'assunzione di non collusione, nessuna entità può scoprire la preferenza dell'elettore contro la sua volontà; viene meno solo la possibilità di impedire che sia l'elettore a rivelarla. Le implicazioni progettuali di questo compromesso sono discusse in WP2.

1.6 Definizioni Formali

Si forniscono di seguito alcune definizioni formali delle proprietà di sicurezza più rilevanti per il sistema. Le definizioni sono accompagnate da una spiegazione in linguaggio naturale per facilitarne la comprensione.

Definizione 1 — Unicità. Sia ε l'insieme degli elettori aventi diritto al voto e sia V l'insieme

dei voti registrati nell'urna al termine della raccolta. Il sistema soddisfa la proprietà di *unicità* se e solo se:

$$\forall e \in \varepsilon : |\{v \in V \mid v \text{ è stato espresso da } e\}| \leq 1.$$

In altre parole: per ogni elettore legittimo, il numero di voti a lui attribuibili nell'urna è al più uno. Un elettore che non ha partecipato contribuisce con zero voti; un elettore che ha partecipato contribuisce con esattamente un voto. Non è ammessa alcuna eccezione.

Definizione 2 — Verificabilità individuale. Sia $e \in \varepsilon$ un elettore che ha espresso il voto $c_e \in C$, e sia r_e la ricevuta crittografica rilasciata al momento del voto. Il sistema soddisfa la proprietà di *verificabilità individuale* per e se e solo se esiste un algoritmo pubblico *Verify* tale che:

$$\text{Verify}(r_e, V) = 1 \iff c_e \in V.$$

In altre parole: ogni elettore, utilizzando esclusivamente la propria ricevuta r_e e l'urna pubblica V , può verificare autonomamente che il proprio voto sia stato correttamente incluso nel conteggio finale. L'algoritmo di verifica è pubblico e non richiede la collaborazione di alcuna entità del sistema.

Definizione 3 — Pseudoanonimato. Siano info_{IdP} l'insieme delle informazioni in possesso dell'Identity Provider — ovvero le identità degli elettori autenticati e le relative autorizzazioni al voto — e info_{AE} l'insieme delle informazioni in possesso dell'Autorità Elettorale — ovvero i voti registrati nell'urna in forma anonima. Il sistema soddisfa la proprietà di *pseudoanonimato*, sotto l'assunzione che IdP e AE non colludano, se e solo se non esiste alcuna funzione f tale per cui valga quanto segue:

$$f(\text{info}_{\text{IdP}}, \text{info}_{\text{AE}}) = (e, c_e) \quad \text{per qualche } e \in \varepsilon,$$

dove $c_e \in C$ è il voto effettivamente espresso dall'elettore e .

In altre parole: nessuna entità — né l'IdP, né l'Autorità Elettorale, né un avversario esterno — è in grado di ricostruire la corrispondenza tra un elettore e la sua preferenza combinando le informazioni disponibili nel sistema. Tale proprietà è strutturalmente garantita dalla separazione dei ruoli: info_{IdP} contiene le identità ma non i voti, info_{AE} contiene i voti ma non le identità. Nessuna delle due, singolarmente, è sufficiente a violare lo pseudoanonimato.

Definizione 4 — Integrità. Sia V_{sub} l'insieme dei voti correttamente sottomessi dagli elettori legittimi durante la finestra di votazione, e sia V_{final} l'insieme dei voti registrati nell'urna al termine della raccolta. Il sistema soddisfa la proprietà di *integrità* se e solo se vale quanto segue:

$$V_{\text{final}} = V_{\text{sub}}.$$

In altre parole: l'insieme dei voti registrati nell'urna al termine della raccolta coincide esattamente con l'insieme dei voti legittimamente sottomessi dagli elettori.

- Nessun voto può essere aggiunto fraudolentemente, quindi ogni voto che si trova nell'urna finale deve provenire dai voti sottomessi: $V_{\text{final}} \subseteq V_{\text{sub}}$.
- Allo stesso modo, nessun voto può essere rimosso fraudolentemente, quindi ogni voto legittimamente sottomesso deve trovarsi nell'urna finale: $V_{\text{sub}} \subseteq V_{\text{final}}$.

Nota. Ovviamente, nessun voto deve essere modificato nel contenuto dopo la sua sottomissione.

Nota. È chiaro che tale proprietà rappresenta l'*integrità ideale* (impossibilità di cambiare i voti).

WP 2 — Soluzione

Sistema di voto elettronico sicuro

Premio “Tesi dell’Anno” dell’Ateneo

2.1 Panoramica del Protocollo

Il protocollo progettato per la conduzione sicura del Premio “Tesi dell’Anno” si articola in quattro fasi sequenziali: *setup*, *autenticazione*, *votazione* e *scrutinio*. L’obiettivo è raggiungere un compromesso ragionevole tra segretezza, verificabilità ed efficienza.

La scelta progettuale fondamentale è la separazione fisica e logica dei ruoli tra IdP e Autorità Elettorale, già dichiarata nel WP1: l’IdP autentica l’elettore ma non vede mai il contenuto del voto; l’AE riceve e registra il voto cifrato ma non conosce l’identità di chi lo ha espresso. Il meccanismo che rende operativa questa separazione è l’uso di un *nonce* casuale firmato: l’IdP, dopo aver autenticato l’elettore, genera un identificatore casuale privo di informazioni identificative e lo firma con la propria chiave privata. L’elettore usa questo token per accedere all’urna senza rivelare la propria identità all’AE.

Gli strumenti crittografici impiegati sono i seguenti:

- **RSA-OAEP** per la cifratura asimmetrica del voto con la chiave pubblica di cifratura dell’Autorità Elettorale, garantendo confidenzialità e non determinismo della scheda.
- **Firme digitali RSA** (schema *hash-and-sign* con SHA-256) per autenticare i messaggi delle entità del sistema e garantire integrità e non ripudio.
- **SHA-256** come funzione di hash crittografica per la costruzione delle ricevute e la verifica dell’integrità delle schede pubblicate.
- **TLS** come protocollo di canale sicuro per tutte le comunicazioni tra le parti, garantendo confidenzialità e autenticazione del canale.
- **PKI e certificati X.509** per la distribuzione autenticata delle chiavi pubbliche di tutte le entità del sistema.

Nota. RSA-OAEP è scelto per la resistenza ai *padding oracle attack*, a differenza di PKCS #1.

Il sistema adotta un *bulletin board* pubblico gestito dall’AE: tutte le schede cifrate vengono pubblicate in tempo reale man mano che vengono ricevute, consentendo la verifica individuale da parte degli elettori. Il bulletin board è *append-only* — nessuna voce può essere rimossa o modificata dopo la pubblicazione — e la sua integrità è garantita da una struttura ad albero (*Merkle Tree*), aggiornata ad ogni inserimento.

Nota. Si assume che il bulletin board sia una fonte pubblica unica e non equivocante; in caso contrario un’AE disonesta potrebbe presentare viste divergenti.

Si riconosce esplicitamente che una soluzione crittograficamente più robusta richiederebbe l’uso di *blind signatures* (per lo pseudoanonimato perfetto) o di cifratura omomorfica (per lo scrutinio senza decifrare le singole schede). Tali strumenti non sono stati trattati nel corso e non vengono implementati; vengono menzionati come possibili estensioni future che eliminerebbero alcune delle limitazioni residue analizzate nel WP3.

2.2 Setup e Distribuzione delle Chiavi

La fase di setup precede l'apertura delle urne e ha lo scopo di inizializzare tutte le componenti crittografiche del sistema. Le operazioni descritte in questa sezione vengono eseguite una volta sola per ciascuna sessione elettorale.

Parametro di sicurezza.

Il sistema adotta un parametro di sicurezza $n = 2048$ bit per le chiavi RSA. Tutte le funzioni di hash utilizzano SHA-256, con output a 256 bit.

Infrastruttura a chiave pubblica (PKI).

Il sistema adotta una *Certification Authority* dedicata all'elezione (CA_{voto}), istituita appositamente per questa sessione elettorale e indipendente da qualsiasi infrastruttura universitaria preesistente. CA_{voto} genera la propria coppia RSA (pk_{CA} , sk_{CA}) e produce un certificato auto-firmato (*self-signed*) che funge da ancora di fiducia dell'intero sistema. Prima dell'apertura delle urne, pk_{CA} viene distribuita fuori banda a tutti gli elettori. CA_{voto} utilizza sk_{CA} per emettere i certificati X.509 delle tre entità operative (IdP, AE, Commissione Accademica). Una volta completata la fase di setup e rilasciati tutti i certificati, sk_{CA} viene portata offline: non è più necessaria durante le fasi operative e mantenerla online costituirebbe un rischio inutile per l'intera infrastruttura.

Emissione della CRL iniziale.

Prima di portare sk_{CA} offline, la CA emette una *Certificate Revocation List* (CRL) inizialmente vuota, firmata con la propria chiave privata:

$$CRL_0 = \text{Sign}_{sk_{CA}}(\text{questoAggiornamento} \parallel \text{prossimoAggiornamento} \parallel [])$$

dove *questoAggiornamento* è il timestamp corrente e *prossimoAggiornamento* è fissato alla scadenza della sessione elettorale. La CRL viene pubblicata sul bulletin board insieme ai certificati delle entità operative.

Aggiornamento della CRL.

Se durante la sessione elettorale un certificato viene compromesso e deve essere revocato, la CA viene riportata online per emettere una CRL aggiornata:

$$CRL_{new} = \text{Sign}_{sk_{CA}}(\text{questoAggiornamento}_{new} \parallel \text{prossimoAggiornamento}_{new} \parallel \{\text{seriale revocato}, \text{timestamp revoca}\})$$

La nuova CRL sostituisce la precedente sul bulletin board (è l'unico campo del bulletin board che può essere modificato). L'AE firma il bulletin board aggiornato con la nuova *Merkle root*.

Politica di verifica (Hard-Fail).

Il sistema adotta una politica di verifica *hard-fail*: qualsiasi entità che non riesca a scaricare o verificare la CRL aggiornata deve rifiutare il certificato corrispondente. Questa scelta è motivata dal *threat model* del WP1: un avversario potrebbe tentare di bloccare l'accesso alla CRL per far accettare un certificato revocato.

Generazione delle chiavi delle entità.

Ciascuna entità del sistema genera autonomamente la propria coppia di chiavi prima dell'inizio della sessione elettorale:

- **Autorità Elettorale (AE):** genera due coppie RSA distinte, in applicazione del principio di separazione delle chiavi: una coppia di firma (pk_{AE}, sk_{AE}) , usata per firmare ricevute, *Merkle root*, messaggio di chiusura e risultato; e una coppia di cifratura (ek_{AE}, dk_{AE}) , usata esclusivamente per cifrare e decifrare le schede. Entrambe le chiavi pubbliche sono certificate dalla CA, con l'estensione X.509 *keyUsage* a distinguerne lo scopo (*digitalSignature* per pk_{AE} , *keyEncipherment* per ek_{AE}). La chiave di firma sk_{AE} resta online durante la votazione, in quanto necessaria per le ricevute e per gli aggiornamenti della *Merkle root*; la chiave di decifrazione dk_{AE} può invece essere mantenuta offline fino allo scrutinio, riducendone la finestra di esposizione. La compromissione di dk_{AE} consentirebbe di decifrare tutte le schede, quella di sk_{AE} di falsificare ricevute e risultati. La separazione evita inoltre il riuso di una stessa coppia RSA per cifratura e firma.
- **Identity Provider (IdP):** genera una coppia RSA (pk_{IdP}, sk_{IdP}) . La chiave pubblica pk_{IdP} viene certificata dalla CA e resa disponibile all'AE prima dell'inizio delle votazioni. La chiave privata sk_{IdP} è utilizzata per firmare i token di autenticazione emessi agli elettori.
- **Commissione Accademica:** genera una coppia RSA $(pk_{CAcomm}, sk_{CAcomm})$ utilizzata esclusivamente per firmare digitalmente la lista ufficiale delle tesi candidate. Tale firma garantisce l'autenticità e l'immodificabilità della lista dei candidati. Ovviamente, anche in tal caso, la chiave pubblica pk_{CAcomm} viene certificata dalla CA mediante apposito certificato X.509.

Pubblicazione della lista dei candidati.

La Commissione Accademica compila la lista ufficiale delle opzioni di voto

$$C = \{c_1, c_2, \dots, c_k, c_\perp\}$$

e la firma digitalmente:

$$\sigma_C = \text{Sign}_{sk_{CAcomm}}(H(C))$$

dove $c_1, \dots, c_k$ sono le tesi e $c_\perp$ è un identificatore riservato a lunghezza fissa, garantito distinto da ogni ID di tesi, che rappresenta la scheda bianca. La coppia (C, σ_C) viene pubblicata sul bulletin board prima dell'apertura delle urne. Chiunque può verificare l'autenticità della lista tramite pk_{CAcomm} .

Inizializzazione del Bulletin Board.

L'AE inizializza il bulletin board come struttura pubblica e *append-only*. Ogni voce inserita nel bulletin board viene incorporata in un albero di Merkle aggiornato incrementalmente: la *Merkle root* corrente è pubblicata e firmata dall'AE ad ogni aggiornamento, consentendo a chiunque di verificare che nessuna voce sia stata rimossa o alterata dopo la pubblicazione.

Canali di comunicazione.

Tutte le comunicazioni tra le parti avvengono su canali protetti da TLS, che garantisce autenticazione del server tramite certificati X.509 e confidenzialità del canale tramite Diffie-Hellman. In particolare:

- Il canale Elettore $\rightarrow$ IdP è protetto da TLS con autenticazione del server (l'IdP presenta il proprio certificato).
- Il canale Elettore $\rightarrow$ AE è protetto da TLS con autenticazione del server (l'AE presenta il proprio certificato di firma).
- Il canale IdP $\rightarrow$ AE (per la condivisione di pk_{IdP} nella fase di setup) avviene fuori banda (*out-of-band*), autenticato tramite certificati.

2.3 Fase di Autenticazione

La fase di autenticazione ha lo scopo di rilasciare a ciascun elettore legittimo un *token di autorizzazione al voto* — una credenziale crittografica monouso che gli permetterà di accedere all'urna nella fase successiva. Il requisito fondamentale di questa fase è che il token non contenga alcuna informazione identificativa sull'elettore: l'Autorità Elettorale deve poter convalidare il voto ricevuto senza che sia possibile risalire al suo destinatario.

Prerequisiti.

Prima dell'apertura delle urne, l'IdP dispone di un registro interno R contenente gli identificativi di tutti gli aventi diritto al voto (studenti iscritti e docenti afferenti all'Ateneo). Il registro è pre-caricato e firmato digitalmente dalla Commissione Accademica per garantirne l'autenticità. Ogni voce in R ha un flag booleano *voted*, inizialmente impostato a *False*.

Inoltre, la Commissione pubblica sul bulletin board, in fase di setup, il numero di aventi diritto $N_{elig} = |R|$ firmato con la propria chiave:

$$\sigma_{elig} = \text{Sign}_{sk_{CAcomm}}(H(N_{elig} \parallel election_id))$$

Questo valore è il limite superiore pubblico e autenticato al numero di token emettibili.

Passo 1 — Apertura del canale sicuro

L'elettore stabilisce una connessione TLS con l'IdP, verificando il certificato X.509 di quest'ultimo tramite pk_{CA} . Prima di procedere, l'elettore scarica la CRL corrente dal bulletin board e verifica che il numero seriale del certificato dell'IdP non sia presente nella lista:

$$serial_{IdP} \notin CRL$$

Se la verifica CRL fallisce, sia perché il certificato è revocato, sia perché la CRL non è raggiungibile (politica *hard-fail*), la connessione viene interrotta e il processo di autenticazione non prosegue.

Nota. Il canale TLS garantisce confidenzialità e autenticazione del server per tutta la durata della sessione.

Passo 2 — Autenticazione dell'elettore

L'elettore si autentica all'IdP tramite le proprie credenziali (es. nome utente e password). L'IdP verifica:

- Che l'identità dell'elettore sia presente nel registro R (l'elettore ha diritto a votare).
- Che il flag *voted* associato a quell'identità sia *False* (l'elettore non ha ancora votato).

Se uno dei due controlli fallisce, la richiesta viene rifiutata e non viene emesso alcun token.

Nota. Le credenziali (password) sono conservate in forma di *hash con salt*, per garantire una maggiore sicurezza in casi di attacco all'IdP.

Passo 3 — Generazione del token

Se entrambi i controlli hanno esito positivo, l'IdP:

1. Genera un *Nonce* casuale $N \leftarrow \{0, 1\}^{256}$, privo di qualsiasi informazione identificativa sull'elettore.

2. Calcola la firma sul token:

$$\sigma_N = \text{Sign}_{sk_{IdP}}(H(N \parallel election_id))$$

dove *election_id* è un identificativo univoco della sessione elettorale corrente, incluso per prevenire attacchi di *replay* tra sessioni diverse.

3. Imposta il flag *voted* dell'elettore a *True* nel registro *R*, prima di inviare il token, per prevenire la sottomissione di richieste duplicate in caso di *race condition*.

Nota. È fondamentale impostare il flag *prima* di inviare il token, per prevenire la sottomissione di richieste duplicate in caso di *race condition*.

- Se il flag viene impostato prima e l'invio del token fallisce → l'elettore è bloccato, ma nessuno può votare due volte.
- Se il flag venisse impostato dopo → un attaccante potrebbe inviare due richieste simultanee prima che il flag venga aggiornato, ricevendo due token validi. Il sistema violerebbe direttamente la proprietà di unicità.

Tuttavia, se il token non arriva per un problema di rete, l'elettore legittimo si ritrova con il flag a *True* ma senza token. Tale scenario va gestito opportunamente, ad esempio con un meccanismo di *recovery*: l'elettore contatta l'IdP su un canale autenticato per richiedere il token già emesso.

Passo 4 — Consegna del token

L'IdP trasmette il token $T = (N, \sigma_N)$ all'elettore attraverso il canale TLS. Il formato del token è il seguente:

$$T = (N, \sigma_N), \quad N \in \{0, 1\}^{256}, \quad \sigma_N \in \{0, 1\}^{2048}$$

L'elettore conserva T localmente fino alla fase di votazione. Il token è monouso e non trasferibile: chiunque ne venga in possesso potrebbe esercitare il diritto di voto al posto del legittimo elettore.

Scelta progettuale — pseudoanonimato del token

Il nonce N è generato in modo puramente casuale e non contiene alcun riferimento all'identità dell'elettore. L'AE, ricevendo il token nella fase successiva, è in grado di verificarne la validità crittografica tramite pk_{IdP} , ma non dispone di alcuno strumento per risalire all'identità di chi lo ha ricevuto. **Il rischio residuo, dichiarato esplicitamente nel WP1, è la collusione tra IdP e AE: un IdP e un'AE colludenti potrebbero tentare di associare il token emesso (conosciuto dall'IdP) con il voto espresso (conosciuto dall'AE).** L'IdP conosce la coppia (identità, N), l'AE conosce la coppia (N , voto). Effettuando una semplice *join* tra le due sul token N , si ottiene banalmente la corrispondenza (identità, voto), violando in maniera diretta lo pseudoanonimato. Tale rischio residuo può essere mitigato mediante l'utilizzo di *blind signatures*, citato come estensione futura.

Diagramma di sequenza.

Per una maggiore comprensione della fase di autenticazione, la Figura 2 riporta il diagramma di sequenza di tale fase, contenente tutti i passi specificati precedentemente.

2.4 Fase di Votazione

La fase di votazione descrive le operazioni che l'elettore compie per compilare e trasmettere la propria scheda, e le operazioni che l'Autorità Elettorale esegue per riceverla, registrarla e rilasciare una ricevuta crittografica.

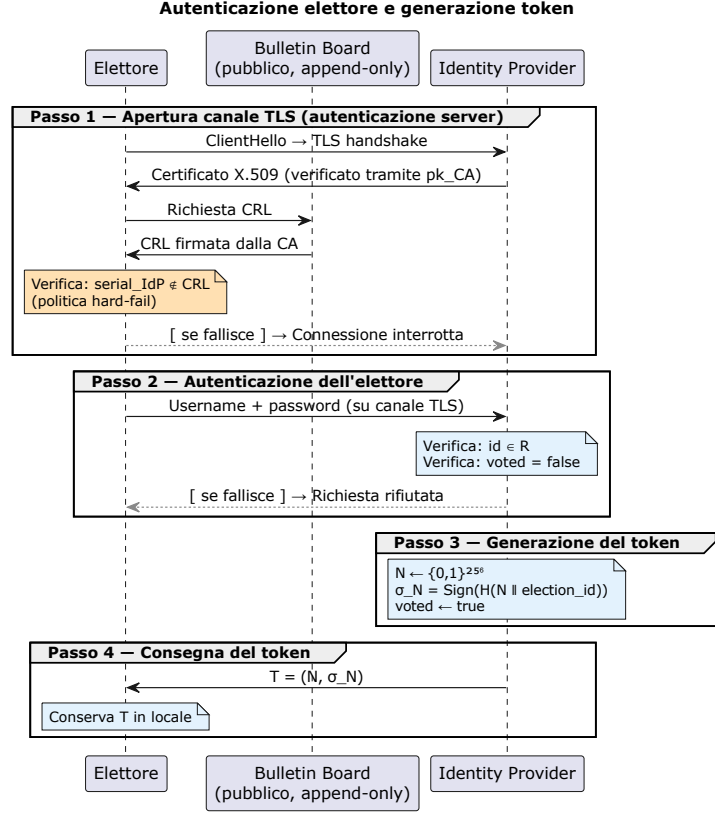


Figura 2: Diagramma di sequenza della fase di autenticazione.

Passo 1 — Preparazione della scheda

L'elettore seleziona la propria opzione di voto $c_v \in C$ (una tesi candidata oppure la scheda bianca $c_\perp$). Genera quindi un nonce personale:

$$r \leftarrow \{0, 1\}^{256}$$

L'elettore costruisce il plaintext m della scheda come concatenazione della preferenza e del nonce:

$$m = c_v \parallel r$$

e cifra la scheda con la chiave pubblica dell'Autorità Elettorale usando RSA-OAEP:

$$ct = \text{Enc}_{ek_{AE}}(m) = \text{Enc}_{ek_{AE}}(c_v \parallel r)$$

Nota. c_v è rappresentato come un identificatore numerico a lunghezza fissa assegnato dalla Commissione Accademica nella lista ufficiale C .

Nota. L'uso di RSA-OAEP garantisce che due cifrature dello stesso messaggio producano *ciphertext* diversi grazie al padding randomizzato, rendendo lo schema CCA-sicuro (quindi anche CPA-sicuro). Il nonce r è un segreto personale dell'elettore: non serve a garantire non determinismo (già RSA-OAEP è non deterministico), ma viene conservato come segreto per aumentare ulteriormente l'entropia del ciphertext.

Passo 2 — Composizione e trasmissione del messaggio di voto

L'elettore compone il messaggio di voto nella seguente forma:

$$M_{vote} = (N, \sigma_N, ct)$$

e lo trasmette all'Autorità Elettorale attraverso il canale TLS. Il messaggio contiene il token di autenticazione ottenuto nella fase precedente (N, σ_N) e il ciphertext della scheda ct .

Prima di trasmettere il messaggio di voto, l'elettore scarica la CRL corrente dal bulletin board e verifica che il numero seriale del certificato dell'AE non sia presente nella lista: $serial_{AE} \notin CRL$. Se la verifica fallisce (politica *hard-fail*), la trasmissione viene interrotta.

Nota. Si noti che M_{vote} non contiene alcun riferimento esplicito all'identità dell'elettore.

Passo 3 — Verifica del token da parte dell'AE

Alla ricezione di M_{vote} , l'AE esegue i seguenti controlli in sequenza:

1. Verifica della firma del token:

$$\text{Vrfy}_{pk_{IdP}}(H(N \parallel election_id), \sigma_N) \stackrel{?}{=} 1$$

Se la verifica fallisce, il messaggio viene scartato: il token non è stato emesso dall'IdP o è stato alterato.

2. Verifica di unicità del Nonce: l'AE controlla che $N \notin U$, dove U è l'insieme dei nonce già utilizzati. Se $N \in U$, il token è stato già speso: si potrebbe trattare di un tentativo di doppio voto. L'AE recupera il ct' associato a N e distingue:

- a. $ct = ct' \rightarrow$ re-invio legittimo dello stesso voto (es. ricevuta persa). L'AE non registra nulla di nuovo, restituisce la ricevuta ρ già emessa e la prova di inclusione di Merkle per ct .
- b. $ct \neq ct' \rightarrow$ tentativo di doppio voto: il messaggio viene scartato e nulla viene registrato.

3. Registrazione del Nonce: l'AE aggiunge N all'insieme U prima di procedere, analogamente alla logica del flag *voted* nell'IdP.

Nota. Prima di procedere alla verifica della firma del token, l'AE controlla che il certificato dell'IdP non sia presente nella CRL: $serial_{IdP} \notin CRL$. Se la verifica fallisce (politica *hard-fail*), il messaggio viene scartato.

Passo 4 — Registrazione sul Bulletin Board

Superati i controlli, l'AE registra la scheda sul bulletin board con la seguente voce:

$$BB_{entry} = (ct, timestamp, H(ct))$$

L'AE aggiorna incrementalmente l'albero di Merkle incorporando la nuova voce e pubblica la nuova *Merkle root* firmata:

$$\sigma_{root} = \text{Sign}_{sk_{AE}}(H(MerkleRoot \parallel timestamp))$$

La firma aggiornata sulla *Merkle root* garantisce che chiunque possa verificare in ogni momento che nessuna voce precedentemente pubblicata sia stata rimossa o alterata.

Nota. Il *timestamp* registrato nella voce BB_{entry} e nella ricevuta è deliberatamente a grana grossa (arrotondato all'ora). Un *timestamp* preciso, essendo pubblico, costituirebbe un canale di correlazione temporale: l'IdP, che conosce l'istante di autenticazione di ciascun elettore, potrebbe da solo correlare identità e scheda — e quindi, dopo la pubblicazione di D, identità e voto — senza alcuna collusione con l'AE. La granularità oraria fa confluire le schede di una stessa finestra temporale in un unico insieme indistinguibile, chiudendo il canale senza alcun costo per le altre proprietà: l'unicità della voce è garantita dal *ciphertext*, non dal *timestamp*.

Passo 5 — Emissione della ricevuta

L'AE calcola e trasmette all'elettore la ricevuta crittografica:

$$\rho = \text{Sign}_{sk_{AE}}(H(ct \parallel timestamp \parallel election_id))$$

La ricevuta è la prova firmata dall'AE che il ciphertext ct è stato correttamente ricevuto e registrato sul bulletin board al tempo $timestamp$.

Passo 6 — Conservazione locale dell'elettore

L'elettore conserva in locale il proprio *fascicolo di verifica*:

$$F = (c_v, r, ct, \rho)$$

Questo fascicolo contiene tutto il necessario per eseguire la verifica individuale nella fase di scrutinio: la preferenza espressa, il nonce, il ciphertext registrato e la ricevuta firmata dall'AE.

Nota sui limiti.

Il sistema non impedisce la coercizione: poiché il ciphertext ct è pubblicato sul bulletin board e la ricevuta ρ ne certifica la presenza, un elettore potrebbe essere costretto da un coercitore a mostrare F per dimostrare la propria scelta. Questo è un limite strutturale di tale soluzione. Nel WP3 verrà analizzato in dettaglio.

Nota. Il sistema non prevede alcun meccanismo di sostituzione o annullamento del voto una volta che la scheda è stata registrata sul bulletin board. Questa è una scelta progettuale deliberata, motivata nel WP1: l'irrevocabilità rafforza la resistenza alla coercizione e semplifica la gestione crittografica dell'urna. Un voto registrato è definitivo.

Diagramma di sequenza.

Per una maggiore comprensione della fase di votazione, la Figura 3 riporta il diagramma di sequenza di tale fase, contenente tutti i passi specificati precedentemente.

2.5 Scrutinio e Pubblicazione del Risultato

La fase di scrutinio ha inizio alla chiusura ufficiale delle urne e comprende la decifrazione delle schede, la validazione dei voti, il conteggio e la pubblicazione del risultato finale in forma verificabile pubblicamente.

Passo 1 — Chiusura delle urne

Allo scadere della finestra temporale di votazione, l'AE smette di accettare nuovi messaggi di voto e pubblica sul bulletin board un messaggio di chiusura firmato:

$$\sigma_{close} = \text{Sign}_{sk_{AE}}(H(\text{"ELEZIONI CHIUSE"} \parallel timestamp_{close} \parallel MerkleRoot_{final}))$$

Il messaggio include la *Merkle root* finale dell'urna, chiarendo lo stato definitivo del bulletin board. Da questo momento, nessuna modifica all'urna è più possibile senza invalidare σ_{close} .

Contestualmente alla chiusura, l'IdP pubblica sul bulletin board il numero totale di token emessi $K = |\{e \in R : voted_e = \text{True}\}|$, firmato:

$$\sigma_{count} = \text{Sign}_{sk_{IdP}}(H(K \parallel election_id))$$

Poiché il flag *voted* impedisce l'emissione di più di un token per voce di R (per un IdP onesto), K coincide con il numero di elettori autenticati.

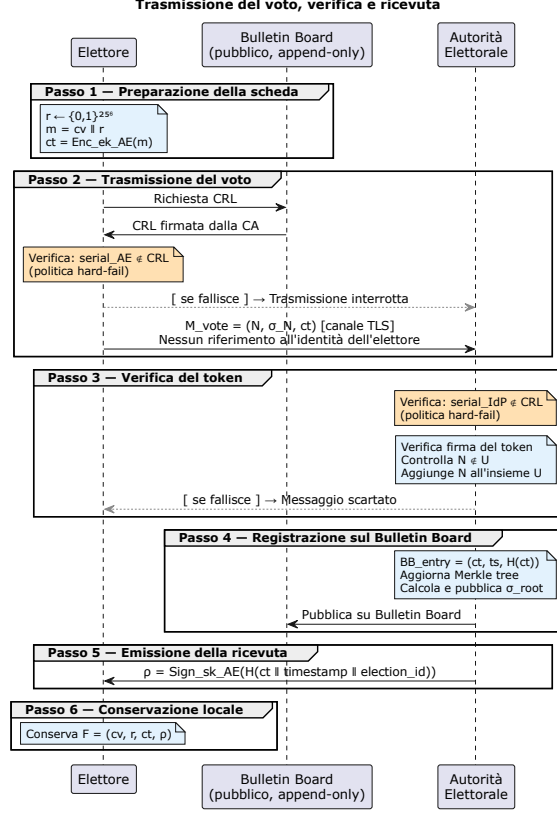


Figura 3: Diagramma di sequenza della fase di votazione.

Passo 2 — Decifrazione delle schede

L'AE decifra ciascun ciphertext ct_i presente sul bulletin board utilizzando la propria chiave privata:

$$(c_{v_i}, r_i) = \text{Dec}_{dk_{AE}}(ct_i)$$

Per ciascuna scheda, l'AE verifica che il risultato della decifrazione sia valido, ovvero che $c_{v_i} \in C$. Le schede per le quali la decifrazione produce un plaintext non appartenente alla lista ufficiale dei candidati ($c_{v_i} \notin C$) vengono marcate come invalide e non contribuiscono al conteggio finale.

Nota. La presenza di schede invalide viene registrata e pubblicata come dato di trasparenza in una lista separata, $I = \{ct_i : c_{v_i} \notin C\}$, in modo tale che il verificatore universale possa controllare che $|BB| = |D| + |I|$: il numero totale di schede presenti sul bulletin board ($|BB|$) dev'essere pari al numero di schede valide ($|D|$) sommato al numero di schede invalide ($|I|$).

Passo 3 — Pubblicazione delle coppie di decifrazione

L'AE pubblica sul bulletin board la lista completa delle coppie:

$$D = \{(ct_i, c_{v_i})\}_{i=1}^m$$

dove m è il numero totale di schede valide ricevute. La pubblicazione di D è il fondamento della verificabilità del sistema: ogni elettore può individuare la propria scheda tramite ct e verificare che la preferenza decifrata corrisponda a quella espressa, mentre chiunque può ricalcolare il conteggio finale a partire da D .

Passo 4 — Calcolo del risultato finale

L'AE calcola il risultato finale come vettore di conteggi per ciascun candidato:

$$T[c_j] = |\{i : c_{v_i} = c_j\}| \quad \forall c_j \in C$$

Nota. Tale vettore contiene, per ciascuna opzione di voto, il numero di voti effettivamente ricevuti.

Nota. $T[c_\perp]$ è il numero di schede bianche. Ovviamente, il vincitore dell'elezione sarà calcolato esclusivamente su $C \setminus \{c_\perp\}$ — la scheda bianca non concorre alla vincita.

Passo 5 — Firma e pubblicazione del risultato

L'AE firma il risultato finale (conteggi per ciascun candidato):

$$\sigma_{result} = \text{Sign}_{sk_{AE}}(H(T[c_1] \parallel T[c_2] \parallel \dots \parallel T[c_k] \parallel T[c_\perp] \parallel \text{timestamp}_{close}))$$

e pubblica la tupla

$$(T, \sigma_{result}, D, \text{MerkleRoot}_{final}, \sigma_{close})$$

come risultato ufficiale dell'elezione. L'inclusione del timestamp di chiusura nella firma lega il risultato a uno specifico momento nel tempo, prevenendo la produzione di risultati antidatati o postdatati.

Verifica universale del risultato.

Chiunque — elettori, candidati, osservatori esterni — può verificare la correttezza del risultato eseguendo i seguenti controlli indipendentemente:

1. Verificare σ_{close} tramite pk_{AE} : conferma che l'urna è stata chiusa correttamente.
2. Verificare la *Merkle root* finale: conferma che il bulletin board non è stato alterato dopo la chiusura (integrità).
3. Ricalcolare il conteggio a partire da D : conferma che il conteggio riportato corrisponde alle coppie pubblicate.
4. Verificare σ_{result} tramite pk_{AE} : conferma che il risultato è stato prodotto dall'AE autentica.

Nota sui limiti — Verifica della correttezza della decifrazione.

Il sistema nella sua forma attuale non fornisce una prova crittografica di correttezza della decifrazione: chiunque può verificare che il conteggio corrisponde alle coppie pubblicate in D , ma non può verificare direttamente che $\text{Dec}_{dk_{AE}}(ct_i) = c_{v_i}$ senza conoscere dk_{AE} .

Diagramma di sequenza.

Per una maggiore comprensione della fase di scrutinio, la Figura 4 riporta il diagramma di sequenza di tale fase, contenente tutti i passi specificati precedentemente.

2.6 Meccanismo di Verifica Individuale e Universale

Questa sezione descrive in modo preciso gli algoritmi che consentono rispettivamente ad ogni singolo elettore di verificare il proprio voto e a qualsiasi osservatore esterno di verificare la correttezza dell'intero scrutinio.

2.6.1 Verifica Individuale

La verifica individuale è eseguita dall'elettore utilizzando esclusivamente il proprio fascicolo di verifica $F = (c_v, r, ct, \rho)$ e i dati pubblici disponibili sul bulletin board.

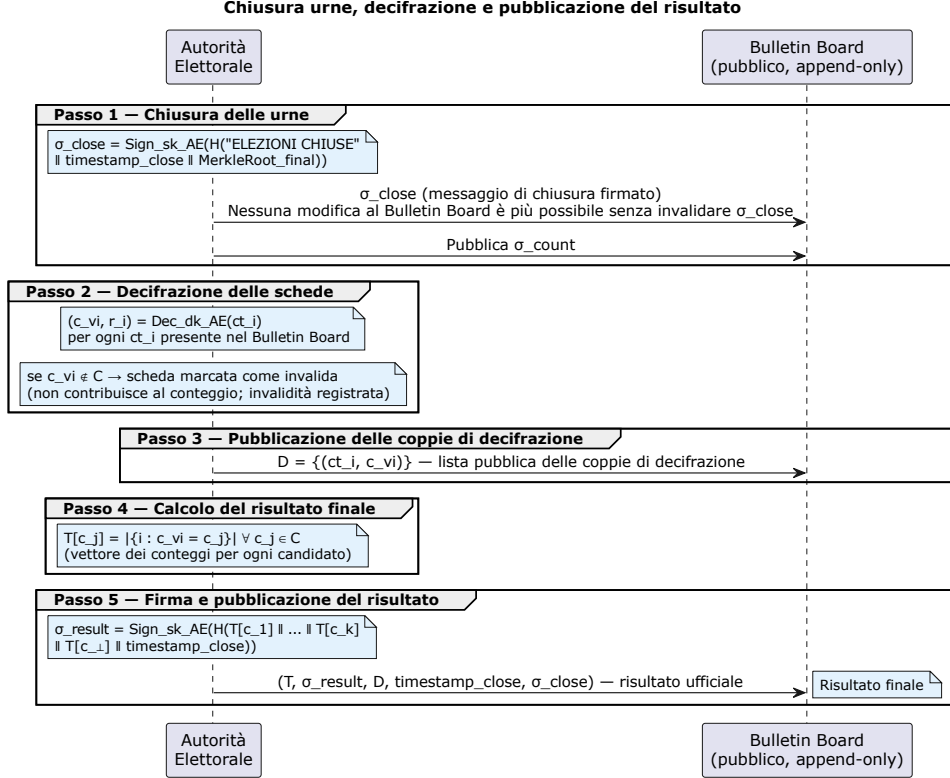


Figura 4: Diagramma di sequenza della fase di scrutinio.

Algoritmo di verifica individuale.

$$\text{VerifyVote}(F, BB, D, pk_{AE})$$

Passo 0 — Verifica CRL

Prima di procedere, l'elettore scarica la CRL corrente dal bulletin board e verifica che il certificato dell'AE non sia revocato: $serial_{AE} \notin CRL$. Se la verifica fallisce (politica *hard-fail*), la procedura di verifica individuale viene interrotta.

Passo 1 — Verifica dell'autenticità della ricevuta

L'elettore verifica che la ricevuta ρ sia stata prodotta dall'AE autentica sul proprio ciphertext:

$$\text{Vrfy}_{pk_{AE}}(H(ct \parallel \text{timestamp} \parallel \text{election_id}), \rho) \stackrel{?}{=} 1$$

Se la verifica fallisce, la ricevuta è contraffatta o non corrisponde al ciphertext conservato.

Passo 2 — Verifica dell'inclusione nel bulletin board

L'elettore verifica che il proprio ciphertext ct sia presente nel bulletin board richiedendo all'AE la prova di inclusione di Merkle (*Proof of Membership*) relativa alla propria scheda. L'elettore verifica:

$$\text{VerifyMerkleProof}(ct, \pi_{ct}, \text{MerkleRoot}_{final}) \stackrel{?}{=} 1$$

dove π_{ct} è il percorso di Merkle fornito dall'AE. Questa verifica garantisce che ct fosse incluso nell'urna al momento della chiusura e che non sia stato rimosso successivamente.

Passo 3 — Verifica del conteggio corretto

L'elettore cerca la propria coppia nella lista di decifrazione pubblica D :

$$\exists (ct_i, c_{v_i}) \in D : ct_i = ct$$

e verifica che il voto decifrato corrisponda alla propria preferenza:

$$c_{v_i} \stackrel{?}{=} c_v$$

Se $c_{v_i} \neq c_v$, il voto è stato conteggiato in modo errato e l'elettore deve segnalare pubblicamente la discrepanza, fornendo il proprio fascicolo F come evidenza.

Nota sui limiti — Verificabilità della decifrazione.

L'elettore non dispone di un meccanismo crittografico per provare a terzi il contenuto del proprio plaintext senza rivelare il *seed* interno di RSA-OAEP, che nelle implementazioni standard non è accessibile all'utente. Questa limitazione implica che, in caso di contestazione, la verifica della correttezza della decifrazione richiederebbe l'intervento di una commissione arbitrale con accesso a dk_{AE} , indebolendo la natura autonoma della verifica.

Diagramma di sequenza.

Per una maggiore comprensione del meccanismo di verifica individuale, la Figura 5 riporta il diagramma di sequenza di tale meccanismo, contenente tutti i passi specificati precedentemente.

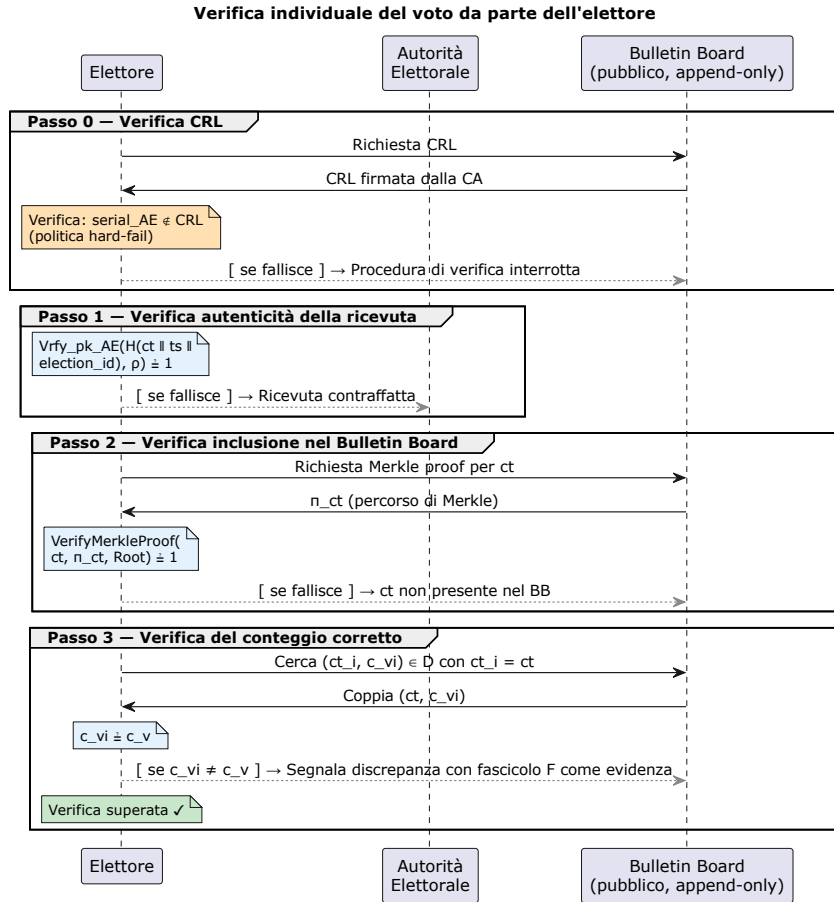


Figura 5: Diagramma di sequenza della fase di verifica individuale.

2.6.2 Verifica Universale

La verifica universale è eseguibile da qualsiasi osservatore esterno — candidati, commissione accademica, enti terzi — utilizzando esclusivamente i dati pubblici del bulletin board. Non richiede l'accesso ad alcuna informazione riservata.

Algoritmo di verifica universale.

$$\text{VerifyElection}(BB, D, T, \sigma_{result}, \sigma_{close}, pk_{AE})$$

Passo 0 — Verifica CRL

Prima di procedere, il verificatore scarica la CRL corrente dal bulletin board e verifica che tutti i certificati impiegati nei controlli successivi (firma dell'AE, IdP e della Commissione) siano validi e non siano revocati: $serial_{AE}, serial_{IdP}, serial_{Comm} \notin CRL$. Se la verifica fallisce (politica *hard-fail*), la procedura di verifica universale viene interrotta.

Passo 1 — Verifica della chiusura dell'urna

$$\text{Vrfy}_{pk_{AE}}(H(\text{"ELEZIONI CHIUSE"} \parallel timestamp_{close} \parallel MerkleRoot_{final}), \sigma_{close}) \stackrel{?}{=} 1$$

Conferma che l'urna sia stata chiusa dall'AE autentica e che la *Merkle root* finale sia certificata.

Passo 2 — Verifica dell'integrità del bulletin board

Il verificatore ricalcola la *Merkle root* a partire da tutte le entry pubblicate nel bulletin board e verifica che coincida con $MerkleRoot_{final}$ certificata in σ_{close} . Questo garantisce che nessuna scheda sia stata aggiunta, rimossa o modificata dopo la chiusura.

Passo 3 — Verifica della consistenza della lista di decifrazione

Il verificatore controlla che ogni ct_i presente in D corrisponda a una entry del bulletin board:

$$\forall (ct_i, c_{v_i}) \in D : ct_i \in BB$$

e che ogni $c_{v_i} \in C$ sia un'opzione di voto valida. Il verificatore controlla inoltre che ogni entry del BB compaia in D o in I , e che $|BB| = |D| + |I|$.

Passo 3-bis — Verifica del bilancio dei voti

Il verificatore verifica le firme σ_{elig} (tramite $pk_{CA_{comm}}$) e σ_{count} (tramite pk_{IdP}), e controlla la catena:

$$m \leq K \leq N_{elig}$$

dove $m = |D|$ è il numero di schede valide. Se la disuguaglianza non è rispettata, l'elezione è respinta: sono state registrate più schede dei token emessi, o emessi più token degli aventi diritto.

In altre parole: il verificatore deve verificare che il numero di schede valide ($m = |D|$) sia al più pari al numero di token emessi (K), che a sua volta dev'essere al più pari al numero di aventi diritto (N_{elig}).

Passo 4 — Verifica del conteggio

Il verificatore ricalcola il conteggio a partire da D :

$$T'[c_j] = |\{i : c_{v_i} = c_j\}| \quad \forall c_j \in C$$

e verifica che $T' = T$.

Passo 5 — Verifica della firma sul risultato

$$\text{Vrfy}_{pk_{AE}}(H(T[c_1] \parallel \dots \parallel T[c_k] \parallel T[c_\perp] \parallel \text{timestamp}_{close}), \sigma_{result}) \stackrel{?}{=} 1$$

Conferma che il risultato pubblicato sia stato prodotto dall'AE autentica e non sia stato alterato.

Diagramma di sequenza.

Per una maggiore comprensione del meccanismo di verifica universale, la Figura 6 riporta il diagramma di sequenza di tale meccanismo, contenente tutti i passi specificati precedentemente.

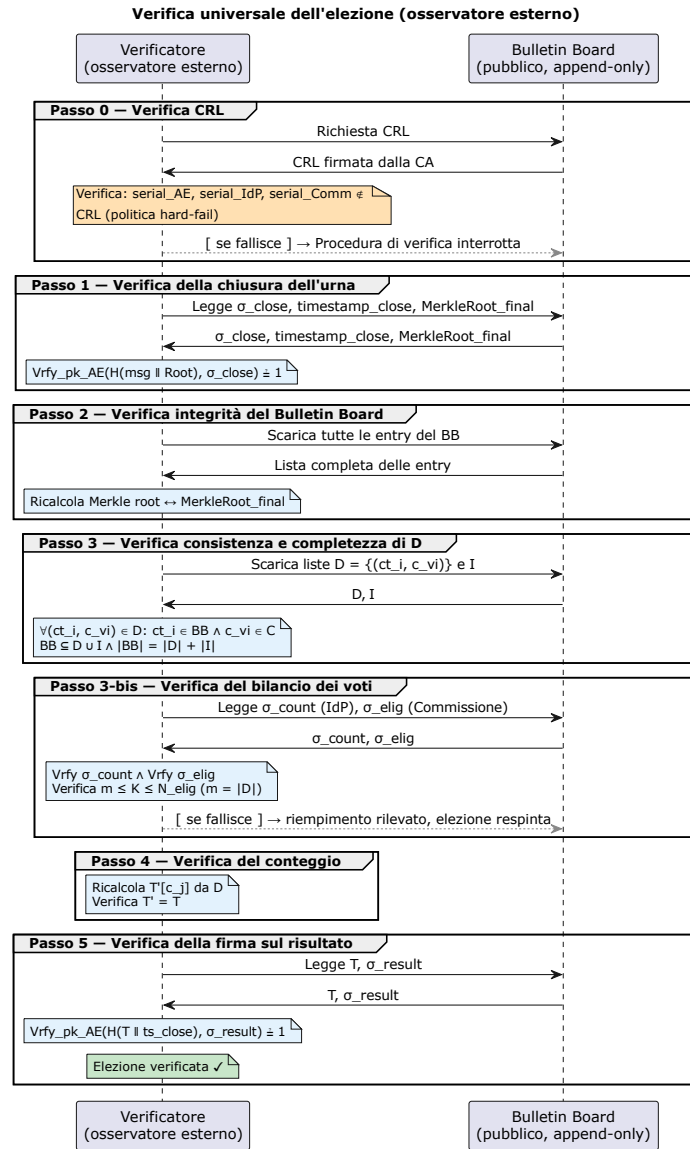


Figura 6: Diagramma di sequenza della fase di verifica universale.

2.7 Riepilogo dei Compromessi Progettuali

Il protocollo progettato rappresenta un compromesso ragionevole tra le proprietà di sicurezza identificate nel WP1, l'efficienza computazionale e gli strumenti crittografici trattati nel corso.

In questa sezione si analizza sistematicamente in che misura ciascuna proprietà è soddisfatta, con quali limitazioni residue e sotto quali assunzioni.

Proprietà	Soddisfatta	Meccanismo	Limitazione residua
Autenticità	Piena	Token firmato dall'IdP, verificato dall'AE + meccanismo di conteggio dei token	Dipende dalla segretezza di sk_{IdP}
Unicità	Piena	Flag <i>voted</i> + insieme U dei nonce usati + conteggio dei token	Race condition mitigata dall'ordine delle operazioni
Segretezza	Parziale	Separazione IdP/AE + nonce casuale	Rischio di <i>linkage</i> dell'identità in caso di collusione
Integrità	Piena	Bulletin board append-only + Merkle tree + firme dell'AE	Dipende dalla segretezza di sk_{AE}
Verif. individuale	Parziale	Ricevuta firmata + Merkle proof + lista D	Impossibile provare la correttezza della decifrazione senza ZKP
Verif. universale	Parziale	Lista D pubblica + ricalcolo del <i>tally</i>	Impossibile provare la correttezza della decifrazione senza ZKP
Disponibilità	Non garantita	—	Attacchi DoS non mitigabili con gli strumenti crittografici del corso
Irrevocabilità	Piena	Assenza deliberata di un meccanismo di revoca	Scelta progettuale consapevole

Tabella 2: Soddisfacimento delle proprietà di sicurezza: meccanismo adottato e limitazione residua per ciascuna proprietà.

Compromesso 1 — Segretezza vs. Efficienza

Il meccanismo di pseudoanonimato adottato — nonce casuale firmato dall'IdP — è computazionalmente efficiente e non richiede strumenti avanzati, ma non garantisce lo pseudoanonimato in senso crittograficamente forte. La soluzione ideale sarebbe l'uso di *blind signatures*: l'elettore preparerebbe un nonce, lo “accecherebbe” crittograficamente prima di inviarlo all'IdP, e l'IdP firmerebbe il nonce senza mai vederne il contenuto. In questo modo l'IdP non potrebbe correlare il token firmato all'elettore nemmeno in caso di collusione con l'AE. Questo strumento non è stato trattato nel corso e non viene implementato; viene indicato come la principale estensione futura del protocollo.

Le blind signature eliminerebbero anche il rischio residuo di manomissione del numero di token (un IdP disonesto che cerca di emettere più token falsificati del previsto), poiché renderebbero il numero di token pubblicamente riconciliabile con la lista dei partecipanti senza violare lo pseudoanonimato.

Compromesso 2 — Verificabilità individuale vs. Receipt Freeness

Il sistema consente a ogni elettore di verificare autonomamente che il proprio voto sia stato incluso e conteggiato correttamente. Tuttavia, poiché la ricevuta ρ certifica la presenza di ct nel bulletin board pubblico, un elettore potrebbe mostrare il proprio fascicolo F a un coercitore per dimostrare come ha votato. Questo viola la proprietà di *receipt-freeness*, ovvero l'impossibilità per l'elettore di provare a terzi la propria scelta. Tale proprietà richiederebbe meccanismi crittografici avanzati non trattati nel corso.

Si riconosce che nel contesto del Premio “Tesi dell'Anno”, dove la comunità accademica è ristretta e le dinamiche di pressione informale sono concrete, questo limite ha un impatto pratico non trascurabile.

Compromesso 3 — Trasparenza dello scrutinio vs. Prova di correttezza della decifrazione

La pubblicazione della lista $D = \{(ct_i, c_{v_i})\}$ garantisce la verificabilità universale del conteggio, ma non fornisce una prova crittografica che ciascuna coppia (ct_i, c_{v_i}) sia il risultato corretto della decifrazione. Chiunque può verificare che il conteggio corrisponda a D , ma non può verificare che D corrisponda alla corretta decifrazione delle schede senza accesso a dk_{AE} . Una soluzione più robusta richiederebbe prove di correttezza della decifrazione (*decryption proofs*), basate su ZKP, indicate come estensioni future.

Compromesso 4 — Centralizzazione della chiave di decifrazione dell'AE

Nel protocollo attuale, dk_{AE} è detenuta da un'unica entità. Un'AE compromessa o disonesta potrebbe decifrare tutte le schede prima della chiusura delle urne, violando la segretezza. Una soluzione più robusta distribuirebbe dk_{AE} tra più entità usando uno schema a soglia (t, n) : la chiave sarebbe ricostruibile solo con la cooperazione di almeno t delle n entità, eliminando il singolo punto di vulnerabilità.

Compromesso 5 — Sicurezza dell'autenticazione vs. Complessità (Estensione Futura)

L'attuale implementazione prevede un'autenticazione tramite credenziali classiche (username e password) su canale TLS. Sebbene le password siano protette tramite *hash e salt*, questo approccio espone teoricamente il sistema ad attacchi di *phishing* o *credential stuffing*. Come estensione futura, in un'architettura di identità federata completa (es. OIDC), si prevede l'imposizione nativa dell'autenticazione a più fattori (MFA) o l'adozione di standard *passwordless* come FIDO2/WebAuthn. Questo azzererebbe il rischio di compromissione degli account, garantendo che solo il reale avente diritto possa ottenere il token di voto dall'IdP.

Compromesso 6 — Centralizzazione del Bulletin Board

Nel protocollo attuale il *bulletin board* è gestito da un'unica entità, l'Autorità Elettorale, che pubblica le schede, costruisce incrementalmente il *Merkle tree* e firma la root a ogni aggiornamento. La correttezza della verifica — sia individuale sia universale — poggia sull'assunzione, dichiarata in 2.1, che il bulletin board sia una fonte pubblica unica e non equivocante. Questa scelta è motivata dal vincolo di efficienza, prioritario fin dal WP1: un registro a entità singola non richiede alcun protocollo di consenso né il coordinamento di più nodi, mantenendo basse la latenza e la complessità operativa. Il rischio residuo è che un'AE disonesta presenti viste divergenti del bulletin board a verificatori diversi: poiché ciascun verificatore convaliderebbe la propria copia rispetto alla σ_{close} ricevuta, l'incongruenza non emergerebbe dal singolo controllo, compromettendo simultaneamente l'integrità e la verificabilità universale. Una soluzione strutturalmente più robusta sostituirebbe il registro centralizzato con un registro append-only distribuito (*Distributed Ledger Technology*, ad es. le *Blockchain*), in cui lo stato del bulletin board è replicato e concordato tra più nodi indipendenti: ciò eliminerebbe il singolo punto di fiducia, rendendo l'equivocazione impossibile senza compromettere la maggioranza dei nodi. Tale soluzione non viene adottata perché introdurrebbe un costo computazionale e di coordinamento incompatibile con il vincolo di efficienza dichiarato in WP1; viene indicata come estensione futura.

WP 3 — Analisi della Sicurezza

Sistema di voto elettronico sicuro

Premio “Tesi dell’Anno” dell’Ateneo

3.1 Obiettivo e Metodologia dell’Analisi

Questo work package analizza la sicurezza del protocollo presentato nel WP2 rispetto al modello di minaccia e alle proprietà di sicurezza definite nel WP1. L’obiettivo è *verificare in che misura ciascuna proprietà è effettivamente raggiunta*, sotto quali assunzioni e con quale rischio residuo, riconducendo la sicurezza del protocollo alle assunzioni crittografiche delle primitive impiegate.

Modello di avversario e assunzioni ereditate dal WP1.

Coerentemente con il WP1, tutti gli avversari sono modellati come algoritmi *probabilistici in tempo polinomiale* (PPT). L’analisi si svolge sotto le seguenti assunzioni di fiducia, già dichiarate nel modello e qui rese esplicite perché ogni affermazione di sicurezza vi è subordinata:

- **Sicurezza delle primitive.** RSA con parametro $n = 2048$ bit è una permutazione one-way; lo schema di firma RSA *hash-and-sign* (PKCS #1 v1.5 con SHA-256) è esistenzialmente infalsificabile sotto attacco a messaggio scelto; RSA-OAEP è CCA-sicuro; SHA-256 è resistente alle collisioni.
- **Ancora di fiducia.** Il certificato auto-firmato di CA_{voto} è distribuito fuori banda in modo autentico e sk_{CA} è mantenuta offline dopo il setup; la PKI e i canali TLS si comportano come specificato.
- **Onestà nelle funzioni assegnate.** Ciascuna entità (IdP, AE, Commissione) è onesta nell’esercizio del proprio ruolo, salvo quando un esperimento ne ipotizza esplicitamente la disonestà o la compromissione.
- **Non collusione IdP / AE.** È l’assunzione cardine dello pseudoanonimato: l’analisi della segretezza è valida *solo* sotto questa ipotesi, il cui venir meno è studiato come limite strutturale.
- **Bulletin board non equivocante.** Il registro pubblico è una fonte unica e coerente per tutti i verificatori; la rilassazione di questa ipotesi è discussa come rischio residuo.

Metodo: esperimenti di sicurezza formali e riduzioni.

Adottiamo lo stesso formalismo usato nel corso per definire la sicurezza delle primitive. Ogni proprietà del protocollo è espressa come un *esperimento* (gioco) $\text{Esp}_{A,\Pi}(n)$ tra uno sfidante e un avversario PPT A , con la struttura consueta: si esegue $\text{Gen}(1^n)$ per generare le chiavi delle entità; A riceve le chiavi pubbliche ed eventuali oracoli; al termine l’esperimento produce un bit, pari a 1 se e solo se A vince. La proprietà è soddisfatta se, per ogni avversario PPT A , la probabilità $\Pr[\text{Esp}_{A,\Pi}(n) = 1]$ è limitata come richiesto: $\leq \text{negl}(n)$ per autenticità, unicità e integrità; $\leq \frac{1}{2} + \text{negl}(n)$ per la segretezza, in stile indistinguibilità. La garanzia è poi dimostrata per *riduzione*: si mostra che un avversario capace di vincere l’esperimento del protocollo permetterebbe di costruire un avversario contro uno degli esperimenti delle primitive viste nel corso — **Sig-forging** per le firme, PubK^{cca} per RSA-OAEP, **Hash-coll** per SHA-256. Dove una proprietà non si riduce ad una primitiva ma poggia su un’assunzione del modello (la non collusione IdP / AE, l’onestà funzionale dell’IdP), ciò è dichiarato esplicitamente.

A ciascun esperimento formale affianchiamo una o più *istanze concrete*: scenari avversariali specifici in cui si riporta l’esito atteso dello specifico esperimento. Le istanze rendono tangibile l’esperimento e ne documentano il comportamento; sono identificate dai codici E1–E9 richiamati nella sezione successiva. Gli esperimenti si dividono in due classi: quelli in cui l’attacco deve fallire (esito atteso $\Pr[\cdot] \leq \text{negl}(n)$: **Proprietà preservata**) e quelli che dimostrano un limite dichiarato, in cui un avversario efficiente vince con certezza ($\Pr[\cdot] = 1$: **Limite confermato**) — non difetti del protocollo, ma conferme formali dei rischi residui già identificati nel WP1.

Una precisazione su “prevenzione” e “rilevazione”.

Diverse proprietà del protocollo non sono garantite in senso *preventivo* (rendere l’attacco impossibile), bensì in senso *rilevativo* (rendere l’attacco sempre individuabile da chi verifica). L’integrità dell’urna e la verificabilità ne sono l’esempio principale: un’AE disonesta *può* tentare di alterare o sopprimere una scheda, ma qualunque tentativo è rilevabile pubblicamente e rende l’elezione respingibile. Questa distinzione è mantenuta esplicita in tutta l’analisi, perché incide direttamente sul significato delle garanzie offerte.

3.2 Analisi delle Proprietà di Sicurezza

Si analizzano di seguito, una per una, le proprietà definite nel WP1. Per ciascuna si indicano il meccanismo che la realizza, gli avversari che la minacciano, l’argomento di sicurezza con la relativa riduzione alle assunzioni crittografiche, gli esperimenti che la validano e il rischio residuo.

Autenticità.

Solo gli aventi diritto possono partecipare al voto. Il meccanismo che la realizza è il token di autorizzazione: l’AE accetta un messaggio di voto solo se la firma del token è verificata, ovvero $\text{Vrfy}_{pk_{IdP}}(H(N \parallel \text{election_id}), \sigma_N) = 1$; in chiusura, il numero di token emessi K , firmato dall’IdP, è confrontato con il numero di aventi diritto N_{elig} , firmato dalla Commissione, nella catena $m \leq K \leq N_{\text{elig}}$. Gli avversari rilevanti sono l’elettore *malintenzionato* (che tenta di votare senza titolo), l’IdP *disonesto* (che ammette soggetti non legittimi) e l’avversario di rete.

L’argomento di sicurezza è una riduzione diretta all’infalsificabilità della firma: per ottenere l’accettazione di un voto senza un token legittimo, un avversario PPT dovrebbe produrre una coppia (N, σ_N) valida su un nuovo messaggio $H(N \parallel \text{election_id})$ senza conoscere sk_{IdP} , ossia realizzare una contraffazione esistenziale che contraddice l’ipotesi di infalsificabilità della firma. L’esperimento **E1** verifica questo punto: un token con firma arbitraria viene respinto, mentre un token legittimo è accettato. Resta da osservare che il legame tra “token valido” e “avente diritto” è demandato all’IdP, che è fidato per questa funzione: la disonestà dell’IdP nell’*ammissione* di non aventi diritto non è contrastata crittograficamente ed è rilevabile solo se l’IdP eccede N_{elig} (catena del bilancio). Il rischio residuo è pertanto duplice: la segretezza di sk_{IdP} (la cui compromissione consentirebbe di coniare token validi a piacere) e la fiducia nell’IdP per il rispetto dei requisiti di eleggibilità entro il limite N_{elig} . *Verdetto*: **Proprietà preservata**, condizionata alla segretezza di sk_{IdP} e all’onestà dell’IdP sull’eleggibilità.

Unicità.

Ciascun avente diritto esprime al più un voto valido (Definizione di unicità, WP1). La proprietà poggia su tre meccanismi ridondanti: il flag *voted* nel registro dell’IdP (al più un token per voce), l’insieme U dei nonce spesi presso l’AE (al più una scheda registrata per nonce) e il conteggio firmato K dei token emessi. L’avversario tipico è l’elettore *malintenzionato* che tenta il doppio voto; secondariamente, un IdP *disonesto* che emetta più token alla stessa voce.

L'argomento è combinatorio e si appoggia all'infalsificabilità della firma. Due voti distinti e validi imputabili allo stesso elettore richiederebbero o due token distinti — impedito dal flag *voted* per un IdP onesto — oppure il riuso di uno stesso token con un ciphertext diverso, che l'AE rifiuta perché $N \in U$ (controllo eseguito *prima* della registrazione, così da neutralizzare le *race condition*). La sola alternativa, riusare un nonce per un nuovo token, equivarrebbe a falsificare una firma su un nuovo N . L'esperimento **E2** conferma il comportamento: il secondo voto con token riusato e scheda diversa è respinto, il re-invio identico è trattato in modo idempotente (l'AE restituisce la ricevuta già emessa, senza aggiungere voci), e nell'urna resta una sola scheda. Il rischio residuo è il medesimo dell'autenticità: la disonestà dell'IdP (più token alla stessa identità) sfugge ai controlli locali e diviene visibile solo se $K > N_{\text{elig}}$. *Verdetto: **Proprietà preservata***, sotto onestà dell'IdP e segretezza di sk_{IdP} .

Segretezza (pseudoanonimato).

Nessuna entità deve poter associare l'identità dell'elettore alla sua preferenza (Definizione di pseudoanonimato, WP1). I meccanismi sono la separazione dei ruoli IdP / AE, il nonce casuale privo di informazione identificativa e la cifratura della scheda con RSA-OAEP sotto ek_{AE} . Gli avversari sono l'IdP da solo, l'AE da sola, l'avversario di rete e — caso critico — la collusione IdP + AE.

L'argomento distingue i due regimi. *Sotto non collusione*, l'informazione si ripartisce in $\text{info}_{\text{IdP}} = \{(\text{identità}, N)\}$ e $\text{info}_{\text{AE}} = \{(N, ct, \text{voto})\}$: l'IdP conosce l'identità ma non il voto; l'AE conosce il voto (dopo lo scrutinio) ma non l'identità. Nessuna funzione delle informazioni di una sola entità ricostruisce la coppia (identità, voto), come richiesto dalla definizione. La riservatezza in transito è inoltre assicurata da TLS verso un avversario di rete passivo. *In presenza di collusione*, invece, l'unione dei due registri consente un *join* sul nonce N che ricostruisce immediatamente (identità, voto): l'esperimento **E8** lo dimostra con una de-anonimizzazione del 100% degli elettori. Si tratta del limite strutturale già dichiarato, non di un difetto implementativo. Concorrono al rischio residuo due ulteriori elementi: la correlazione di metadati di rete da parte di un avversario attivo (l'osservazione ravvicinata delle connessioni di un elettore prima all'IdP e poi all'AE), non eliminata crittograficamente; e la centralizzazione di dk_{AE} , che consente a un'AE disonesta di decifrare le schede prima della chiusura. *Verdetto: **Proprietà preservata in modo parziale***, subordinato alla non collusione, all'onestà dell'AE riguardo alla decifrazione anticipata e alla segretezza delle chiavi.

Integrità.

I voti registrati non possono essere alterati, soppressi o introdotti fraudolentemente (Definizione di integrità, WP1). I meccanismi sono il bulletin board *append-only*, l'albero di Merkle con *root* firmata incrementalmente dall'AE, il messaggio di chiusura σ_{close} che vincola la *root* finale, le ricevute ρ e la catena di bilancio $m \leq K \leq N_{\text{elig}}$. L'avversario principale è l'AE disonesta; secondariamente l'avversario di rete.

L'argomento riconduce ciascuna manomissione a una violazione di SHA-256 o della firma. *Alterazione*: modificare una voce pubblicata cambia il digest della foglia e quindi la *root*; per presentare una *root* coerente con la firma esistente servirebbe una collisione di SHA-256. L'esperimento **E4** mostra che la *root* ricalcolata dopo la modifica non coincide con quella firmata. *Soppressione*: rimuovere una scheda fa sì che la prova di inclusione di Merkle dell'elettore vittima, valutata rispetto alla *root* finale, non possa più esistere; l'esperimento **E5** ne dà conferma, e la rilevazione avviene in sede di verifica individuale. *Introduzione fraudolenta*: l'inserimento di schede prive di un token valido viola il bilancio $m \leq K$, come mostra l'esperimento **E6** ($m = 85 > K = 80$), e fa respingere l'elezione in sede di verifica universale. L'integrità è qui garantita in senso *rilevativo*: l'AE non è resa incapace di tentare la manomissione, ma ogni tentativo è pubblicamente individuabile. Il rischio residuo è la segretezza di sk_{AE} (la cui compromissione permetterebbe di

ri-firmare una storia alterata in modo coerente) e l'assunzione di bulletin board non equivocante: un'AE che presentasse viste divergenti, ciascuna coerente con la propria σ_{close} , eluderebbe il controllo del singolo verificatore. *Verdetto: **Proprietà preservata***, in senso rilevativo, sotto segretezza di sk_{AE} , resistenza alle collisioni di SHA-256 e non equivocazione del registro.

Verificabilità individuale.

Ogni elettore può verificare autonomamente l'inclusione e il corretto conteggio del proprio voto (Definizione di verificabilità individuale, WP1). Il meccanismo è il fascicolo $F = (c_v, r, ct, \rho)$ unito ai dati pubblici: l'elettore verifica l'autenticità della ricevuta, l'inclusione di ct tramite prova di Merkle e la corrispondenza $(ct, c_v) \in D$. L'avversario è l'AE *disonesta* che tenti di sopprimere o conteggiare erroneamente la scheda.

L'argomento si riduce all'infalsificabilità della firma (ricevuta ρ e σ_{root}) e alla resistenza di SHA-256 (prova di Merkle): una ricevuta contraffatta, un'omissione dall'urna o un voto attribuito a un candidato diverso da quello espresso sono tutti rilevati dall'elettore. Esiste tuttavia un limite preciso e importante: l'elettore *rileva* una discrepanza ma non può *provarla a terzi* per la fase di decifrazione. Poiché il seed interno di RSA-OAEP non è accessibile, l'elettore non può dimostrare in modo autonomo che il proprio ct decifri davvero a c_v ; in caso di contestazione servirebbe una commissione arbitrale con accesso a dk_{AE} . Inoltre, lo stesso fascicolo che abilita la verifica abilita la coercizione: l'esperimento **E9** mostra che, dati F e la lista pubblica D , un coercitore prova come ha votato l'elettore (tensione con la *receipt-freeness* dichiarata nel WP1). Si segnala infine una dipendenza di disponibilità, non di correttezza: la prova di inclusione π_{ct} è fornita dall'AE, che potrebbe rifiutarsi di consegnarla. *Verdetto: **Proprietà preservata in modo parziale***: piena in fase di rilevazione, limitata nella prova della corretta decifrazione.

Verificabilità universale.

Chiunque può verificare la correttezza del risultato finale (WP1). Il meccanismo è la pubblicazione dell'urna, delle liste D e I e del risultato T , con i controlli di verifica universale: firma di chiusura, ricalcolo della *root*, consistenza $|BB| = |D| + |I|$, bilancio $m \leq K \leq N_{elig}$, ricalcolo del conteggio $T' = T$ e firma sul risultato. L'avversario è l'AE *disonesta*.

L'argomento è analogo a quello dell'integrità: ogni discordanza tra il risultato dichiarato e i dati pubblici è ricalcolabile da chiunque e riconducibile a una falsificazione di firma o a una collisione di hash. Il riempimento è intercettato dal bilancio (**E6**); un conteggio falso è intercettato dal ricalcolo $T' = T$. Il limite è il medesimo della verifica individuale: il verificatore conferma che " T è coerente con D ", ma non che " D è la decifrazione corretta dell'urna", in assenza di prove di decifrazione (ZKP). Un'AE che decifri onestamente ma pubblichi un D con attribuzioni errate produrrebbe un conteggio internamente coerente, e solo gli elettori direttamente danneggiati potrebbero accorgersene (verifica individuale). Vale inoltre l'assunzione di non equivocazione del bulletin board. *Verdetto: **Proprietà preservata in modo parziale***: piena sulla coerenza aritmetica del conteggio, limitata sulla correttezza della decifrazione.

Disponibilità.

Il sistema deve essere raggiungibile per tutta la finestra di voto. Come dichiarato nel WP1, la mitigazione degli attacchi DoS esula dagli strumenti crittografici del corso e non è affrontata: nessun esperimento la riguarda, trattandosi di un non-obiettivo. Si osserva anzi che la politica *hard-fail* sulla CRL introduce un sacrificio *deliberato* di disponibilità a favore di autenticità / integrità: l'irraggiungibilità della CRL provoca il rifiuto del certificato, per impedire che un avversario forzi l'accettazione di un certificato revocato bloccando l'accesso alla lista. *Verdetto: **Non garantita***, come previsto dal modello.

Irrevocabilità.

Un voto registrato è definitivo. È una scelta progettuale: l'assenza di un meccanismo di revoca rafforza la resistenza alla coercizione e semplifica la gestione dell'urna. Il tentativo di un elettore di rivotare per sostituire la propria preferenza ricade nel controllo di unicità (insieme U), già validato da **E2**. *Verdetto: **Proprietà preservata***, per costruzione.

3.3 Analisi per Profilo di Avversario

La sezione precedente ha adottato un punto di vista *centrato sulle proprietà*. Qui si assume il punto di vista complementare, *centrato sull'avversario*: per ciascun profilo della matrice delle minacce del WP1 si descrivono le capacità, il raggio d'azione effettivo, la risposta del protocollo e gli esperimenti che ne documentano il comportamento. I due punti di vista sono volutamente ridondanti: insieme coprono ogni cella della matrice delle minacce.

Identity Provider disonesto.

Dispone del pieno controllo dell'autenticazione e conosce la corrispondenza (identità, N). I suoi obiettivi sono l'ammissione fraudolenta di non aventi diritto, la soppressione di autorizzazioni e il tracciamento. L'ammissione di un non avente diritto *non* è contrastabile crittograficamente, poiché l'IdP è fidato per la funzione di eleggibilità; diviene però rilevabile se l'IdP emette più token degli aventi diritto, perché la catena $m \leq K \leq N_{\text{elig}}$ — con K firmato dall'IdP e N_{elig} firmato dalla Commissione — viene violata (esperimento **E6**). Analogamente, l'emissione di più token a una stessa identità attacca l'unicità ma emerge solo se $K > N_{\text{elig}}$. Da solo, l'IdP *non* può violare la segretezza: conosce (identità, N) ma non il voto, che è cifrato sotto ek_{AE} ; il tracciamento del contenuto è possibile soltanto in collusione con l'AE. Il rischio residuo è quindi la segretezza di sk_{IdP} e la fiducia funzionale sull'eleggibilità, entrambi già dichiarati nel modello.

Autorità Elettorale disonesta.

Detiene dk_{AE} e sk_{AE} e conosce la corrispondenza (N , ct , voto). Può tentare di alterare il conteggio, sopprimere o introdurre schede e di violare la segretezza. Le tre manipolazioni dell'urna sono tutte rilevabili in senso rilevativo: l'alterazione invalida la *root* firmata (**E4**), la soppressione fa fallire la prova di inclusione della vittima (**E5**), l'introduzione di schede senza token viola il bilancio (**E6**). Sul fronte della segretezza, l'AE conosce legittimamente i voti ma non le identità, dunque da sola non de-anonimizza; la centralizzazione di dk_{AE} le consentirebbe però di decifrare le schede *prima* della chiusura, minando la segretezza temporale (non l'anonimato). I due rischi residui specifici sono la compromissione di sk_{AE} — che permetterebbe di ri-firmare una storia alterata coerente — e l'*equivocazione* del bulletin board, ossia la presentazione di viste divergenti a verificatori diversi, ciascuna internamente coerente con la propria σ_{close} .

Collusione IdP + Autorità Elettorale.

È lo scenario più critico per la segretezza e l'unico che la compromette in modo diretto e irrimediabile. Unendo $\text{info}_{IdP} = \{(\text{identità}, N)\}$ e $\text{info}_{AE} = \{(N, \text{voto})\}$, un semplice *join* sul nonce N ricostruisce la corrispondenza (identità, voto): l'esperimento **E8** ottiene una de-anonimizzazione del 100%. Nessun meccanismo del protocollo previene questo attacco, che corrisponde al limite strutturale dichiarato fin dal WP1 ed è eliminabile solo introducendo *blind signatures* (come estensione futura). Si osservi che la collusione non peggiora le altre proprietà rispetto al caso della sola AE disonesta: integrità e verificabilità restano governate dagli stessi meccanismi pubblici.

Elettore malintenzionato.

Dispone di risorse standard. Il doppio voto è bloccato dalla combinazione flag *voted* / insieme

U (E2); il tentativo di votare con un token forgiato è respinto dalla verifica di firma (E1); il riuso di un token di una sessione precedente è neutralizzato dal binding su *election_id* (E3); la revoca non è prevista, e i tentativi di rivotare ricadono nel controllo di unicità. L'unico "successo" possibile per questo avversario è dimostrare a terzi la propria preferenza (E9): ciò non costituisce però una minaccia contrastata, bensì il limite di *receipt-freeness* accettato consapevolmente nel WP1.

Eavesdropper / Man-in-the-Middle.

Un avversario *passivo* non ricava il legame identità-voto dal traffico applicativo, protetto da TLS e da RSA-OAEP; resta esposta la sola correlazione di *metadati* (tempi e indirizzi delle connessioni a IdP e AE), che è un canale laterale non crittografico e non pienamente eliminabile con gli strumenti del corso. Un avversario *attivo* è contrastato dall'autenticazione del server in TLS (certificati X.509 verificati con pk_{CA} e politica *hard-fail* sulla CRL): l'interposizione richiederebbe una chiave privata certificata, e l'alterazione dei messaggi in transito viola l'integrità del canale. Va segnalato un punto sottile: la firma del token σ_N copre $H(N \parallel election_id)$ ma *non* il ciphertext ct ; il legame "questa scheda appartiene a questo token" è stabilito dall'AE al primo utilizzo del nonce e protetto in transito da TLS.

Avversario DoS.

Esula dall'ambito crittografico del progetto, come dichiarato nel WP1, e non è oggetto di esperimenti. Si nota soltanto che la politica *hard-fail* sulla CRL, scelta a tutela di autenticità e integrità, offre per costruzione una leva di *denial of service* mirato (bloccare l'accesso alla CRL induce il rifiuto dei certificati): è un compromesso esplicito tra sicurezza e disponibilità.

3.4 Esperimenti di Sicurezza

Coerentemente con il formalismo del corso, ciascuna proprietà è qui formulata come un esperimento $\text{Esp}_{A,\Pi}(n)$ e la relativa garanzia è dimostrata per *riduzione* a uno degli esperimenti delle primitive — Sig-forged per le firme, PubK^{cca} per RSA-OAEP, Hash-coll per SHA-256. A ogni esperimento si affiancano le *istanze concrete* E1–E9, con l'azione dell'avversario e l'esito atteso. Gli esperimenti Vote-Forge, Vote-Unique, Vote-Anon (nella sua componente di riservatezza della scheda) e Tally-Forge appartengono alla classe in cui l'attacco deve fallire; la componente di linkage di Vote-Anon sotto collusione e l'esperimento Receipt dimostrano invece, con probabilità 1, i due limiti strutturali dichiarati nel WP1.

Esperimento 1 — $\text{Vote-Forge}_{A,\Pi}(n)$ (Autenticità).

1. Il setup esegue $\text{Gen}(1^n)$ e genera le chiavi di tutte le entità; A riceve le chiavi pubbliche ed *election_id*.
2. A ha accesso a un oracolo $\text{Auth}_{sk_{IdP}}(\cdot)$ che, su identità a sua scelta, restituisce token validi (N, σ_N) ; sia Q l'insieme dei nonce così ottenuti.
3. A produce un messaggio di voto $M^* = (N^*, \sigma^*, ct^*)$ e lo sottopone all'AE.
4. L'output è 1 se e solo se l'AE accetta M^* e $N^* \notin Q$.

Sicurezza. $\Pr[\text{Vote-Forge}_{A,\Pi}(n) = 1] \leq \text{negl}(n)$.

Riduzione a Sig-forged. La sicurezza si dimostra mostrando che un eventuale avversario A capace di vincere $\text{Vote-Forge}_{A,\Pi}(n)$ potrebbe essere sfruttato per costruire un avversario B in

grado di vincere l'esperimento $\text{Sig-forge}_{B, \Pi_{\text{Sig}}}(n)$. B partecipa a **Sig-forge**, riceve la chiave pubblica dello schema di firma e la fornisce ad A usandola come pk_{IdP} . Da questo momento, B risponde alle query di A inoltrandole al proprio oracolo di firma per ottenere le firme valide sul token $H(N \parallel \text{election_id})$. Quando A termina il suo attacco e restituisce un messaggio contraffatto M^* con un nuovo nonce $N^* \notin Q$ e una firma valida, B si limita a estrarre la coppia *token-firma* $(H(N^* \parallel \text{election_id}), \sigma^*)$ e ad emetterla come propria falsificazione. Poichè $N^* \notin Q$, si ha la garanzia che quel messaggio non era mai stato firmato dall'oracolo. Di conseguenza, B vince l'esperimento esattamente quando lo vince A . Poichè la probabilità di vincere **Sig-forge** è assunta come trascurabile, lo è necessariamente anche quella di vincere **Vote-Forge**: $\Pr[\text{Vote-Forge}] \leq \Pr[\text{Sig-forge}] \leq \text{negl}(n)$. La corrispondenza “token valido $\Rightarrow$ avente diritto” resta affidata all'IdP per onestà funzionale ed è limitata dalla catena $K \leq N_{\text{elig}}$.

E1 — Forgery del token

istanza di Vote-Forge

Azione. A sottopone un token con nonce arbitrario e firma σ^* scelta a caso (non emessa dall'oracolo).

Esito atteso. Respinto (“firma del token non valida”); il token legittimo di riferimento è accettato.

Verdetto. **Proprietà preservata.**

E3 — Replay tra sessioni

istanza di Vote-Forge

Azione. A ripresenta un token emesso con $\text{election_id} = 2025$ a un'AE che opera con $\text{election_id} = 2026$.

Esito atteso. Respinto: l'AE verifica la firma su $H(N \parallel 2026)$, mentre il token è firmato su $H(N \parallel 2025)$; il nonce risulta così associato a un messaggio mai firmato per la sessione corrente.

Verdetto. **Proprietà preservata.** Il binding su election_id riconduce il replay a una sottoistanza di Vote-Forge.

Esperimento 2 — $\text{Vote-Unique}_{A, \Pi}(n)$ (Unicità).

1. Il setup esegue $\text{Gen}(1^n)$; A impersona un avente diritto e interagisce con IdP e AE secondo il protocollo.
2. A sottomette uno o più messaggi di voto.
3. L'output è 1 se e solo se, al termine, l'urna contiene due schede valide *distinte*, entrambe coperte da token validi riconducibili a un medesimo avente diritto.

Sicurezza. $\Pr[\text{Vote-Unique}_{A, \Pi}(n) = 1] \leq \text{negl}(n)$.

Argomento e riduzione. Per ottenere due schede contate servono due token con nonce distinti: l'insieme U rifiuta il riuso di un nonce con ct diverso, e il re-invio identico è idempotente. Il flag *voted* concede al più un token per identità; un secondo token richiederebbe o la sovraemissione di un IdP disonesto (esclusa per onestà funzionale e comunque colta da $K \leq N_{\text{elig}}$) o la forgiatura di un token su un nonce fresco, che si riduce a **Sig-forge**. I nonce sono campionati a 256 bit, dunque due token distinti coincidono solo con probabilità trascurabile. Pertanto, $\Pr[\text{Vote-Unique}] \leq \Pr[\text{Sig-forge}] \leq \text{negl}(n)$.

Azione. Primo voto (C001); secondo voto con lo stesso token ma scheda diversa (C002); infine re-invio identico del primo voto.

Esito atteso. Primo voto accettato; secondo respinto ($N \in U$, ct diverso); re-invio idempotente con restituzione della ricevuta già emessa; schede nell'urna = 1.

Verdetto. **Proprietà preservata.** La registrazione di N in U prima dell'elaborazione neutralizza la *race condition*.

Esperimento 3 — $\text{Vote-Anon}_{A,\Pi}(n)$ (Segretezza / pseudoanonimato).

1. Il setup esegue $\text{Gen}(1^n)$; ad A è concessa la vista di una sola entità tra IdP e AE (un'entità disonesta isolata, sotto non collusione).
2. A sceglie due aventi diritto e_0, e_1 e una preferenza c ; si sceglie un bit uniforme $b \in \{0, 1\}$ e si esegue il protocollo facendo esprimere a e_b la preferenza c , in modo che la sola differenza osservabile sia l'associazione identità-voto.
3. A produce un bit b' ; l'output è 1 se e solo se $b' = b$.

Sicurezza. $\Pr[\text{Vote-Anon}_{A,\Pi}(n) = 1] \leq \frac{1}{2} + \text{negl}(n)$, sotto l'assunzione di non collusione.

Riduzione a PubK^{cca} (riservatezza della scheda). Il ciphertext ct non deve rivelare la preferenza: un avversario che distinguesse i due casi a partire da ct fornirebbe un distinguitore contro l'esperimento PubK^{cca} di RSA-OAEP. Questa componente si riduce dunque alla CCA-sicurezza di RSA-OAEP.

Assunzione (assenza di linkage). Con la vista di una sola entità, info_{IdP} non contiene il voto e info_{AE} non contiene l'identità; poiché il nonce è casuale e privo di informazione identificativa, A non fa meglio di $\frac{1}{2}$. Questa parte non è una riduzione ad una primitiva ma poggia sull'assunzione architetturale di non collusione.

Limite (collusione). Se ad A è concessa la vista di *entrambe* le entità, il *join* sul nonce N dà $\Pr[\text{Vote-Anon}_{A,\Pi}(n) = 1] = 1$: l'avversario vince l'esperimento con probabilità unitaria e, quindi, il protocollo non soddisfa la segretezza sotto collusione. È il limite strutturale del WP1.

Azione. Si cifra due volte lo stesso voto (C001) e, come controprova, due volte un plaintext identico.

Esito atteso. Ciphertext sempre distinti; entrambi decifrano a C001; i ciphertext differiscono anche con plaintext identico (padding OAEP non deterministico).

Verdetto. **Proprietà preservata.** Manifestazione della CCA-sicurezza di RSA-OAEP: schede uguali non sono correlabili sul bulletin board.

Azione. Le due entità uniscono i registri ed eseguono il *join* sul nonce N su cinque elettori.

Esito atteso. Identità de-anonimizzate: 5/5 (100%); l'esperimento è vinto con probabilità 1.

Verdetto. Limite confermato. Conferma formale del rischio residuo; la rimozione richiede *blind signatures* (estensione futura).

Esperimento 4 — Tally-Forge_{A,Π}(n) (Integrità e Verificabilità).

1. Il setup esegue $\text{Gen}(1^n)$; un insieme V di schede è sottomesso da elettori onesti con token validi. A controlla l'AE.
2. A pubblica il *transcript* $(BB, \text{root}, \sigma_{\text{close}}, D, T, \sigma_{\text{result}})$ e le prove di Merkle.
3. L'output è 1 se e solo se la verifica universale e quella individuale (per ogni elettore onesto) hanno successo e il conteggio T differisce da quello reale di V (per aggiunta, rimozione o alterazione di una scheda registrata).

Sicurezza. $\Pr[\text{Tally-Forge}_{A,\Pi}(n) = 1] \leq \text{negl}(n)$, sotto l'assunzione di bulletin board non equivocabile.

Riduzione a Hash-coll e Sig-forge. Alterare o rimuovere una scheda mantenendo coerente la *root* firmata richiede una collisione di SHA-256 (Hash-coll); introdurre schede prive di token viola la catena $m \leq K \leq N_{\text{elig}}$, con K ed N_{elig} firmati (falsificarli è Sig-forge); falsificare σ_{close} , σ_{result} o una ricevuta ρ è Sig-forge. Quindi vincere Tally-Forge implica vincere Hash-coll oppure Sig-forge ($\leq \text{negl}(n)$).

Limite (correttezza della decifrazione). L'esperimento cattura aggiunta, rimozione e alterazione a livello di ciphertext, ma non un'AE che decifri onestamente e pubblichi un D con attribuzioni errate: un tale *transcript* è internamente coerente (T corrisponde a D) e supera la verifica universale; solo l'elettore danneggiato lo rileva con la verifica individuale. Chiudere questo varco richiederebbe prove di decifrazione (ZKP), indicate come estensione futura.

E4 — Manomissione di una scheda

istanza di Tally-Forge

Azione. L'AE sostituisce il ciphertext di una voce già pubblicata, aggiornandone l'hash.

Esito atteso. La *root* ricalcolata dal verificatore non coincide più con quella firmata; la prova di inclusione, valida prima dell'attacco, fallisce.

Verdetto. Proprietà preservata. Presentare una *root* coerente richiederebbe una collisione di SHA-256.

E5 — Soppressione di un voto

istanza di Tally-Forge

Azione. L'AE rimuove la scheda di un elettore e ricostruisce urna e albero senza di essa.

Esito atteso. La scheda non è più presente nell'urna finale; la verifica individuale non trova alcuna prova di inclusione valida rispetto alla *root* finale.

Verdetto. Proprietà preservata, in senso rilevativo: l'omissione è sempre individuabile dall'elettore danneggiato.

E6 — Ballot stuffing

istanza di Tally-Forge

Azione. Con $N_{\text{elig}} = 100$ e $K = 80$ token emessi, l'AE inietta 5 schede prive di token, portando m a 85.

Esito atteso. Scenario onesto: $80 \leq 80 \leq 100$, bilancio soddisfatto. Dopo lo stuffing: $m = 85 > K = 80$, bilancio violato, elezione respinta.

Verdetto. **Proprietà preservata**, in senso rilevativo.

Esperimento 5 — $\text{Receipt}_{A,\Pi}(n)$ (**Receipt-freeness** — **LIMITE**).

1. Il setup esegue $\text{Gen}(1^n)$; un elettore onesto vota c_v e ottiene il fascicolo $F = (c_v, r, ct, \rho)$. Dopo lo scrutinio è pubblicata la lista D .
2. Il coercitore A riceve F e i dati pubblici e produce una predizione $\hat{c}$ della preferenza dell'elettore.
3. L'output è 1 se e solo se $\hat{c} = c_v$.

Risultato. Esiste un avversario efficiente con $\Pr[\text{Receipt}_{A,\Pi}(n) = 1] = 1$: tramite la catena $\rho \rightarrow ct \rightarrow D$ l'avversario è in grado di risalire alla preferenza espressa dall'elettore. Il protocollo *non* soddisfa la receipt-freeness, che richiederebbe $\Pr[\cdot] \leq 1/|C| + \text{negl}(n)$: l'avversario non dovrebbe poter fare di meglio che tirare ad indovinare tra $|C|$ possibili alternative.

E9 — Coercizione tramite il fascicolo

istanza di Receipt (LIMITE)

Azione. Il coercitore, ricevuto F , verifica l'autenticità della ricevuta su ct , individua ct sul bulletin board e legge in D il voto associato.

Esito atteso. Ricevuta autentica; ct presente nel bulletin board; D associa $ct \rightarrow C002$, coincidente con il voto reale. Prova riuscita.

Verdetto. **Limite confermato.** Conseguenza diretta della verificabilità individuale.

Proprietà prive di esperimento.

Due proprietà non danno luogo a un esperimento. La *disponibilità* è un non-obiettivo dichiarato: la mitigazione degli attacchi DoS esula dagli strumenti crittografici del corso. L'*irrevocabilità* non richiede un esperimento dedicato: è imposta dagli stessi meccanismi di unicità (**Vote-Unique**), poiché ogni tentativo di revocare o sostituire un voto già registrato ricade nel controllo dell'insieme U (istanza E2).

3.5 Compromessi ed Estensioni Future

L'analisi ha individuato alcuni limiti residui che il protocollo, con i soli strumenti trattati nel corso, non elimina. Si elencano qui le estensioni che li supererebbero, indicando per ciascuna quale proprietà rafforzerebbe e a quale costo. Nessuna di esse è adottata nel WP2 perché richiede primitive non trattate nel corso oppure introduce un costo — quasi sempre di efficienza o di coordinamento — incompatibile con i vincoli dichiarati nel WP1; vengono pertanto indicate come direzioni di sviluppo future.

- **Blind signatures** eliminerebbero il linkage da collusione (la vulnerabilità di **Vote-Anon**), rendendo l'IdP incapace di correlare il token all'identità: segretezza $\uparrow$, ma efficienza e complessità del protocollo di emissione $\downarrow$.
- **Schema a soglia** (t, n) su dk_{AE} rimuoverebbe il singolo punto di decifrazione, impedendo a un'AE isolata di decifrare prima della chiusura: robustezza della segretezza $\uparrow$, ma coordinamento di n entità e cerimonia di chiave $\downarrow$.
- **Prove di decifrazione a conoscenza zero (ZKP)** chiuderebbero il varco di correttezza della decifrazione di Tally-Forge, rendendo D verificabile senza fiducia nell'AE: verificabilità $\uparrow$, ma costo computazionale e concettuale $\downarrow$.

- **Bulletin board su Distributed Ledger (DLT)** eliminerebbe l'assunzione di non equivocazione su cui poggia Tally-Forge, replicando lo stato fra più nodi: integrità e verificabilità ↑, ma protocollo di consenso e latenza ↓.
- **Autenticazione MFA / FIDO2–WebAuthn** rafforzerebbe l'anello umano dell'autenticità contro *phishing* e *credential stuffing*: autenticità ↑, ma attrito d'uso ↓.

3.6 Tabelle Riassuntive

Si raccolgono qui i due quadri di sintesi dell'analisi. La Tabella 3 riassume gli esiti degli esperimenti con la relativa riconduzione formale; la Tabella 4 aggiorna il prospetto di soddisfacimento delle proprietà del WP2, ora corredato dall'esperimento e dalla riduzione (o assunzione) che ne fonda il giudizio.

Cod.	Proprietà	Avversario	Riconduzione	Esito
E1	Autenticità	Elettore (forgery)	Sig-forges	Preservata
E3	Autenticità (freshness)	Elettore (replay)	Sig-forges + binding <i>election_id</i>	Preservata
E2	Unicità	Elettore (doppio voto)	Sig-forges + insieme U	Preservata
E7	Segretezza	Eavesdropper	PubK ^{cca} (RSA-OAEP)	Preservata
E8	Segretezza	Collusione IdP+AE	Assunzione non collusione	Limite
E4	Integrità	AE (manomissione)	Hash-coll	Preservata
E5	Integr./Verif. ind.	AE (soppressione)	Hash-coll	Preservata
E6	Verif. universale	AE (riempimento)	Sig-forges (bilancio)	Preservata
E9	Receipt-freeness	Coercitore	Limite strutturale	Limite

Tabella 3: Esiti degli esperimenti di sicurezza e relativa riconduzione formale agli esperimenti dei primitivi del corso.

3.7 Conclusioni

Sintesi delle garanzie.

L'analisi conferma il quadro dichiarato nel WP2 e lo fonda formalmente. Autenticità, unicità, integrità e irrevocabilità sono garantite in modo *pieno*, ciascuna riconducibile alla sicurezza di una primitiva del corso (Sig-forges, Hash-coll) sotto assunzioni esplicite. Segretezza e verificabilità (individuale e universale) sono garantite in modo *parziale*, con un limite ben circoscritto in ciascun caso. La disponibilità, infine, non è garantita per scelta di perimetro, essendo la difesa dal DoS estranea agli strumenti crittografici trattati.

Rischi residui, in ordine di criticità.

1. **Collusione IdP+AE** — l'unico attacco che compromette direttamente la segretezza (Vote-Anon vinto con probabilità 1, istanza **E8**); risolvibile solo con *blind signatures*.
2. **Correttezza della decifrazione** — la verificabilità garantisce la coerenza fra conteggio e lista D , ma non che D sia la decifrazione corretta dell'urna; richiede prove a conoscenza zero.

Proprietà	Esito	Esperimento	Riduzione / Assunzione	Rischio residuo
Autenticità	Piena	Vote-Forge (E1, E3)	Sig-forgery	Segretezza di sk_{IdP} ; onestà dell'IdP sull'eleggibilità
Unicità	Piena	Vote-Unique (E2)	Sig-forgery + insieme U	Onestà dell'IdP ($K \leq N_{elig}$)
Segretezza	Parziale	Vote-Anon (E7, E8)	PubK ^{cca} + non collusione	Linkage sotto collusione IdP+AE
Integrità	Piena	Tally-Forge (E4, E5, E6)	Hash-coll + Sig-forgery	Segretezza di sk_{AE} ; non equivocazione del BB
Verif. individuale	Parziale	Tally-Forge (E5)	Sig-forgery + Hash-coll	Correttezza della decifrazione (richiede ZKP)
Verif. universale	Parziale	Tally-Forge (E6)	Sig-forgery + Hash-coll	Correttezza della decifrazione (richiede ZKP)
Disponibilità	Non garantita	—	—	Attacchi DoS fuori dall'ambito del corso
Irrevocabilità	Piena	Vote-Unique (E2)	—	Scelta progettuale consapevole

Tabella 4: Soddisfacimento delle proprietà di sicurezza dopo l'analisi del WP3, con l'esperimento che lo valida, la riduzione o l'assunzione che lo fonda e il rischio residuo. Conferma e formalizza il prospetto del WP2.

3. **Equivocazione del bulletin board centralizzato** — un'AE potrebbe presentare viste divergenti, eludendo il controllo del singolo verificatore; risolvibile con un registro distribuito.
4. **Coercizione (assenza di receipt-freeness)** — conseguenza accettata della verificabilità individuale (Receipt, istanza **E9**).
5. **Segretezza delle chiavi** — sk_{IdP} , sk_{AE} e dk_{AE} sono assunzioni trasversali la cui compromissione invaliderebbe le rispettive riduzioni.
6. **Disponibilità** — esposta ad attacchi DoS, fuori ambito.

Assunzioni critiche.

L'intero edificio di sicurezza poggia su un insieme dichiarato di assunzioni: la sicurezza delle primitive (RSA, PKCS #1, OAEP, SHA-256), la non collusione fra IdP e AE, l'onestà funzionale dell'IdP sull'eleggibilità, la segretezza delle chiavi private e la non equivocazione del bulletin board. La trasparenza su queste assunzioni — più che la loro assenza, impossibile in questo contesto — è il vero indicatore di solidità del modello.

Considerazione finale.

Il protocollo realizza un compromesso coerente e difendibile fra segretezza, verificabilità ed efficienza, adeguato al contesto del Premio "Tesi dell'Anno". I suoi limiti non sono difetti occulti, ma scelte esplicite e circoscritte da assunzioni chiare; per ciascuno è identificata la strada di superamento, rimandata a un'estensione futura che ecceda il perimetro didattico del corso.

WP 4 — Implementazione e Prestazioni

Sistema di voto elettronico sicuro

Premio “Tesi dell’Anno” dell’Ateneo

Codice sorgente. L'intero codice sorgente dell'implementazione descritta in questo work package è disponibile nella seguente cartella Drive condivisa:

https://drive.google.com/drive/folders/1FSIiEpFYdf1hsI_g9JpcqbEw9U0YxeDr?usp=drive_link

4.1 Obiettivo e ambiente di simulazione

Questo work package implementa il protocollo progettato nel WP2 in un ambiente simulato, secondo il modello e le proprietà definite nel WP1. Coerentemente con quanto consentito dalla traccia, non si è sviluppata un'applicazione mobile né un'infrastruttura elettorale reale: il sistema è una singola applicazione stand-alone in cui tutte le entità (CA, Commissione Accademica, Identity Provider, Autorità Elettorale, Bulletin Board ed Elettori) vivono nello stesso processo come oggetti distinti che si scambiano messaggi.

La scelta di una simulazione in-process a moduli — anziché di processi separati comunicanti via rete — è deliberata: concentra l'attenzione sulla correttezza crittografica del protocollo e sul flusso dei messaggi, che sono l'oggetto del progetto, evitando la complessità accessoria di socket, serializzazione e gestione della concorrenza reale, non rilevante ai fini didattici. La separazione dei ruoli prevista dal modello è comunque mantenuta in modo rigoroso a livello di codice: ogni entità è una classe che custodisce le proprie chiavi private e non espone all'esterno informazioni che, secondo il modello, non dovrebbe conoscere (in particolare, l'Autorità Elettorale non ha alcun accesso al registro degli elettori dell'IdP).

4.2 Strumenti e librerie

L'implementazione è in Python (sviluppata e testata con Python 3.13) e si appoggia a un'unica dipendenza esterna per la crittografia, la libreria `cryptography` (`pyca/cryptography`), utilizzata durante i laboratori nel corso. Tutti gli strumenti utilizzati sono quelli previsti dal WP2 e trattati nel corso.

La libreria copre i tre livelli necessari al protocollo: le primitive crittografiche (cifratura RSA-OAEP, firme RSA, SHA-256) e — aspetto non secondario — una gestione reale della PKI X.509 e delle *Certificate Revocation List*. Quest'ultimo punto è una scelta qualificante: la PKI non è simulata in modo astratto, ma realizzata con certificati X.509 autentici emessi e firmati da una CA dedicata, dotati delle estensioni standard, e con CRL effettivamente firmate e verificate. Per le funzioni di hash e per la derivazione delle password si è usata anche la libreria standard `hashlib`. L'interfaccia grafica è realizzata con Tkinter, parte della libreria standard, in modo da non introdurre ulteriori dipendenze; essa è un puro strato di presentazione e non contiene logica crittografica.

Una precisazione sulle firme.

Per la cifratura RSA si usa OAEP, mentre per la firma RSA si usa PSS. Lo schema PSS calcola internamente l'hash del messaggio, quindi firmare un messaggio con PSS+SHA-256 equivale alla scrittura $\text{Sign}(H(m))$ usata nel WP2. La verifica dei certificati e delle CRL X.509 avviene invece con il padding PKCS #1 v1.5, che è lo standard del formato X.509 ed è indipendente dallo schema di firma usato a livello applicativo.

4.3 Architettura del codice e mappatura con il protocollo

L'organizzazione del codice rispecchia direttamente gli attori del modello: ad ogni entità corrisponde un modulo. Questa scelta rende immediato il confronto tra documentazione e implementazione e riflette nella struttura stessa del software la separazione dei ruoli. Le primitive crittografiche sono raccolte in un unico modulo condiviso `crypto_utils`, così che la crittografia risieda in un solo punto e il resto del codice si legga come una sequenza di operazioni di alto livello (cifra, firma, verifica).

Modulo	Entità / ruolo	Riferimento WP2
<code>crypto_utils</code>	Primitive: RSA-OAEP, firme RSA, SHA-256, Merkle tree	2.1
<code>pki</code>	CA dedicata, certificati X.509, CRL, verifica hard-fail	2.2
<code>bulletin_board</code>	Bulletin board pubblico append-only + Merkle tree	2.1, 2.2
<code>commission</code>	Commissione Accademica (lista candidati, registro, N_{elig})	2.2, 2.3
<code>idp</code>	Identity Provider (autenticazione, token = nonce firmato)	2.3
<code>authority</code>	Autorità Elettorale (raccolta voti, ricevute, scrutinio)	2.4, 2.5
<code>voter</code>	Elettore (votazione e verifica individuale)	2.4, 2.6.1
<code>verifier</code>	Verifica universale (osservatore esterno)	2.6.2
<code>election</code>	Orchestratura delle quattro fasi del protocollo	2.1

Tabella 5: Mappatura tra moduli del codice ed entità del modello, con il riferimento alla sezione del WP2 che ciascun modulo realizza.

Il bulletin board è un oggetto unico condiviso da tutte le entità, in linea con l'assunzione del WP2 di una fonte pubblica non equivocabile. La classe di orchestratura crea le entità, esegue il setup e mette a disposizione un'interfaccia ordinata per le quattro fasi (setup, autenticazione, votazione, scrutinio).

4.4 Scelte implementative principali

Setup e infrastruttura a chiave pubblica.

Il setup crea una CA dedicata all'elezione, con un certificato self-signed che funge da ancora di fiducia; la chiave pubblica della CA è distribuita "fuori banda" a elettori e verificatori (nella simulazione, consegnata direttamente all'atto della creazione di ciascun oggetto). Il modello di fiducia del WP2, che prevede di portare offline la chiave privata della CA dopo il setup, è reso dal fatto che tale chiave è confinata all'interno dell'oggetto CA e utilizzata unicamente per emettere o aggiornare la CRL.

La CA emette i certificati X.509 delle entità operative valorizzando l'estensione **keyUsage** in modo da distinguere lo scopo di ciascuna chiave: firma digitale per le chiavi di firma di IdP, AE e Commissione, cifratura per la chiave con cui si cifrano le schede. Vengono inoltre pubblicati sul bulletin board, prima dell'apertura delle urne, la lista dei candidati firmata dalla Commissione (con l'identificatore riservato alla scheda bianca), il numero di aventi diritto firmato e una CRL iniziale vuota.

La verifica dei certificati adotta la politica hard-fail prevista dal WP2: ogni volta che un'entità usa un certificato altrui, ne controlla la firma della CA, la validità temporale e l'assenza dalla CRL; se la CRL è mancante o non verificabile, il certificato viene comunque rifiutato. Questo impedisce a un avversario di far accettare un certificato revocato semplicemente bloccando l'accesso alla lista di revoca.

Il canale TLS previsto dal WP2 non è instaurato come connessione di rete reale, ma modellato come canale sicuro. L'autenticità delle chiavi pubbliche, che è l'aspetto crittograficamente rilevante, è comunque garantita dalla PKI X.509 effettivamente implementata.

Autenticazione e token.

Il token rilasciato dall'IdP è un nonce casuale di 256 bit firmato, privo di qualsiasi informazione identificativa, esattamente come nel WP2. La firma lega il nonce all'identificativo della sessione elettorale (*election_id*), in modo da impedire il riuso di un token in una sessione diversa. Le password degli elettori sono conservate nel registro solo come *hash con salt*; la password in chiaro non viene mai memorizzata.

Una scelta implementativa importante riguarda l'ordine delle operazioni a tutela dell'unicità: il flag che marca l'elettore come "già votante" viene impostato *prima* di consegnare il token. In questo modo, anche di fronte a richieste ravvicinate, non è possibile ottenere due token validi per la stessa identità; il prezzo è che un elettore il cui token si perdesse per un guasto resterebbe bloccato, scenario gestito da una funzione di recupero che ri-consegna il token già emesso su canale autenticato.

Votazione e ricezione del voto.

L'elettore costruisce il plaintext della scheda come concatenazione della preferenza e di un nonce personale, e lo cifra con la chiave pubblica di cifratura dell'AE tramite RSA-OAEP, ottenendo un ciphertext non deterministico. La preferenza è codificata come un identificatore numerico a lunghezza fissa, scelta che semplifica e rende deterministica la decifrazione (i primi byte del plaintext sono sempre l'opzione di voto) e che riflette la natura a lista chiusa dell'elezione.

Bulletin board e albero di Merkle.

La natura append-only del bulletin board è imposta a livello di interfaccia: la classe espone soltanto operazioni di aggiunta, e l'area pubblica degli artefatti rifiuta la sovrascrittura di una chiave già pubblicata. L'unico artefatto aggiornabile è la CRL, per sua natura una lista che può crescere. L'integrità è garantita da un albero di Merkle sulle schede, la cui radice è firmata dall'AE ad ogni inserimento.

Per semplicità e leggibilità, l'albero di Merkle viene ricalcolato da zero ad ogni operazione, con costo lineare nel numero di schede. Il verificatore universale ricalcola la radice in modo indipendente e la confronta con quella firmata, mentre la prova di inclusione consente all'elettore di verificare la presenza della propria scheda senza scaricare l'intera urna.

Scrutinio.

Alla chiusura, l'AE pubblica un messaggio di chiusura firmato che "congela" la *Merkle root* finale, e l'IdP pubblica il numero di token emessi, firmato. L'AE decifra quindi ciascuna scheda e ne

valida la preferenza: le schede il cui plaintext non corrisponde a un'opzione ammessa, o la cui decifrazione fallisce, sono raccolte in una lista separata di schede invalide, pubblicata come dato di trasparenza. Vengono poi pubblicate la lista delle coppie (ciphertext, preferenza), il vettore dei conteggi e la firma sul risultato. La separazione tra schede valide e invalide consente al verificatore di controllare che il loro numero copra esattamente le voci dell'urna.

Verifica individuale e universale.

La verifica individuale è svolta dall'elettore usando il solo fascicolo di verifica e i dati pubblici, senza collaborazione di alcuna entità: controlla l'autenticità della ricevuta, l'inclusione della propria scheda tramite prova di Merkle confrontata con la root finale, e la corrispondenza tra il proprio voto e quanto pubblicato nella lista di decifrazione. La verifica universale è realizzata come una funzione che opera esclusivamente sui dati pubblici del bulletin board e sulla chiave pubblica della CA, e che restituisce l'esito di ciascun controllo separatamente — firma di chiusura, ricalcolo della Merkle root, consistenza delle liste, bilancio dei voti, ricalcolo del conteggio, firma sul risultato — oltre all'esito complessivo. Esporre i singoli esiti, anziché un solo verdetto, rende l'analisi più trasparente e facilita la diagnosi in caso di anomalia.

4.5 Validazione funzionale

Oltre allo scenario regolare (autenticazione, voto, scrutinio, verifica individuale di tutti gli elettori e verifica universale con esito positivo), la validazione comprende una serie di controlli negativi che dimostrano il corretto rifiuto degli abusi: secondo tentativo di autenticazione dello stesso elettore, doppio voto con token riusato e scheda diversa, riconoscimento del re-invio identico come legittimo, token con firma contraffatta, autenticazione su certificato revocato (politica hard-fail) e ricevuta non firmata dall'AE che fa fallire la verifica individuale. Tutti i test, positivi e negativi, danno l'esito atteso.

4.6 Prestazioni

Le misure di prestazione coprono le tre dimensioni richieste dalla traccia: il costo delle singole operazioni crittografiche, la dimensione dei messaggi scambiati e l'andamento dei tempi al variare dell'elettorato.

Nota. Essendo una simulazione in-process, non c'è rete reale e, quindi, i tempi di interazione sono pressochè nulli.

Sul fronte delle primitive, l'unica operazione costosa è la generazione di una coppia di chiavi RSA-2048 (decine di millisecondi), eseguita però una sola volta per entità in fase di setup. Le operazioni ricorrenti del protocollo sono molto rapide: cifratura e verifica di firma sono nell'ordine dei centesimi di millisecondo, decifrazione e firma sotto il millisecondo, l'hashing trascurabile.

I messaggi sono compatti e di dimensione sostanzialmente fissa, dominata dalla lunghezza delle firme e dei ciphertext RSA-2048 (256 byte ciascuno). Il messaggio di voto completo è di poche centinaia di byte, e la prova di inclusione di Merkle cresce solo in modo logaritmico con il numero di schede.

La fase di voto cresce linearmente con il numero di elettori mantenendo un costo per elettore sostanzialmente costante (circa due millisecondi a scheda); lo scrutinio cresce anch'esso linearmente; le latenze di verifica restano molto basse — frazioni di millisecondo — anche per elettorati di alcune centinaia di unità. Il sistema risulta dunque adeguato ai vincoli di efficienza dichiarati fin dal WP1.

Messaggio	Dimensione
Token (nonce + firma)	288 byte
Scheda cifrata (RSA-OAEP)	256 byte
Messaggio di voto completo	544 byte
Ricevuta	256 byte
Prova di Merkle (100 schede, 7 hash)	224 byte

Tabella 6: Dimensione dei principali messaggi scambiati dal protocollo.

N elettori	Voto/elett.	Scrutinio	Verif. ind.	Verif. univ.
10	1,94 ms	11 ms	0,12 ms	0,32 ms
50	1,89 ms	43 ms	0,14 ms	0,36 ms
100	1,88 ms	85 ms	0,15 ms	0,41 ms
200	1,93 ms	169 ms	0,20 ms	0,52 ms

Tabella 7: Andamento dei tempi al variare dell'elettorato: costo di voto per elettore, scrutinio complessivo, verifica individuale e verifica universale.

4.7 Assunzioni e limiti dell'implementazione

I limiti dell'implementazione coincidono con i compromessi già dichiarati e analizzati nel WP2 e nel WP3; qui se ne richiama la traduzione concreta nel codice.

- **Canale TLS astratto.** I canali sono modellati come sicuri e non vengono instaurate connessioni TLS reali; l'autenticità delle chiavi resta garantita dalla PKI X.509 implementata.
- **Non collusione IdP / AE.** Lo pseudoanonimato dipende, come nel modello, dall'assunzione che le due entità non colludano: è il rischio residuo strutturale del sistema.
- **Centralizzazione della chiave di decifrazione e del bulletin board.** La chiave di decifrazione è detenuta da un'unica entità e il registro pubblico è a entità singola; le estensioni future indicate nel WP3 (schema a soglia e registro distribuito) non sono implementate.
- **Assenza di prova di correttezza della decifrazione.** Coerentemente con il WP2, non viene fornita una prova a conoscenza zero che le coppie pubblicate siano la decifrazione corretta delle schede; è indicata come estensione futura.
- **Merkle tree ricalcolato per intero.** Per leggibilità si è preferita la ricostruzione completa ad ogni operazione, con costo lineare, anziché una struttura incrementale.